

17.1.3 等价谓词重写

PostgreSQL 和 MySQL 都支持等价谓词重写的优化技术。

所谓谓词重写，就是找出与原谓词等价且效率高的其他谓词，这样把原执行效率低的谓词改用效率高的谓词并重写 SQL 语句，从而提高 SQL 的整体执行效率。

多数谓词实现的都是局部环境下表达受限的语义，其对 SQL 语句的作用范围不够大，即其影响不够大，所以等价谓词重写属于局部优化技术。

不是所有的等价谓词重写技术都属于局部，这是相对的。如“OR 重写并集规则”，虽然是局部的条件子句，但是可能影响整个查询语句。如果查询优化器能够把局部优化技术和全局优化技术结合起来，评估花费、考虑其利弊，优化的效果可能会更好，但这势必对查询优化器的实现提出更高的要求，查询优化器势必更复杂。

对于等价谓词重写技术的实现，PostgreSQL 和 MySQL 都是在局部范围内进行的。

PostgreSQL 和 MySQL 对等价谓词重写技术的支持情况如表 17-3 所示。

表 17-3 等价谓词重写对照表

规 则	谓词原型	转换后的形式	
		PostgreSQL	MySQL
LIKE 规则	name LIKE 'abc%'	不支持 (name)::text ~ 'abc%':text	不支持 'test'.test_like.name like 'abc%'
BETWEEN 和 AND 规则	BETWEEN 10 AND 20	支持 (sno >= 10) AND (sno <= 20)	不支持 'test'.test_like.sno between 10 and 20
IN 转换 OR 规则	age IN (8,12)	不支持	不支持 'test'.test_like.age in (8,12)
IN 转换 ANY 规则	age IN (8,12,21)	支持 age = ANY ({'8,12,21'}::integer[])	不支持 'test'.test_like.age in (8,12,21)
OR 转换 ANY 规则	age=8 OR age=12 OR age=21	不支持 (age = 8) OR (age = 12) OR (age = 21)	不支持 (test.test_like.age = 8) or (test.test_like.age = 12) or (test.test_like.age = 21))
ALL/ANY 转换集函数规则	sno > ANY (10,2*5+3,sqrt(9))	语法不支持	语法不支持
NOT 规则	NOT(age!=8) NOT(age!=sno) NOT(age=sno) NOT(age<sno) NOT(age>sno)	支持 (age = 8) (age = sno) (age < sno) (age >= sno) (age <= sno)	支持 (test.test_like.age = 8) (test.test_like.sno = test.test_like.age) (test.test_like.age <> test.test_like.sno) (test.test_like.age >= test.test_like.sno) (test.test_like.age <= test.test_like.sno)

(续)

规 则	谓词原型	转换后的形式	
		PostgreSQL	MySQL
OR 重写并集规则	(name='abc' and age>15) OR age>18	不支持 (((name)::text = 'abc'::text) AND (age > 15)) OR (age > 18)	不支持 ((`test`.`test_like`.`name` = 'abc') and (`test`.`test_like`.`age` > 15)) or (`test`.`test_like`.`age` > 18)

17.1.4 条件化简

从格式上看，条件子句有一元操作符、二元操作符、多元操作符。通过析取（AND）与合取（OR）操作，把各个表达式连接起来，而表达式可简单可复杂（复杂如嵌套子查询）。但是，如果从关系代数的角度去看，条件子句无非是由与选择操作或者连接操作相关的表达式构成，这就给出一个思路——分解条件子句：选择操作，归到单个表上做过滤条件，优化意味着做选择操作下推；连接操作，归到相关的表上，为表连接提供依据。

表达式如果是一元运算符，则和二元运算相比，格式上会显得很简单，容易先得出结果；如果是相等运算中的 filed=const 格式，则比其他运算更易利用索引，且元组更少（选择率低的可能性更大）；如果是范围运算，则应尽可能利用索引；如果表达式可以求值，则先求值有利于整个条件子句的计算，比如析取表达式中前部的运算结果为 FALSE 则后面的运算不必进行。所以，条件子句的计算中充满了优化点。条件子句优化的本质是：尽早推知运算的结果以有利于对元组数进行计算，使得根据代价估算模型（元组数是重要的计算依据）可以准确地推演出最优查询执行计划。

条件判断如果能利用约束，则可以简化一些表达式。如非空约束有助于在条件表达式中判断对应列所在的表达式的值（如 a 列上有主键索引，则条件 a IS NULL 为 FALSE），MySQL 较好地利用了这一点，PostgreSQL 却缺少这样的方式（这样的方式归属于语义优化的范畴）。

在多表连接的过程中，查询优化器可以找出多表连接的最优查询执行计划，这意味着多个表的最优的连接次序被确定。如果根据表的连接次序确定析取条件的优先判断次序，存在加速判断的可能（处于表达式后面的条件可能不用判断了）。MySQL 在这个方面进行了条件的排序优化（对条件子句按表的连接次序进行排序，多表连接在前的表，条件子句中与此表相关的条件靠前）。

所以，从整体上看，在条件子句优化方面，MySQL 略胜 PostgreSQL。

PostgreSQL 和 MySQL 对条件子句优化技术的支持情况如表 17-4 所示。

表 17-4 条件子句优化对照表

规 则	表达式原型	转换后的表达式形式	
		PostgreSQL	MySQL
把 HAVING 子句并入 WHERE 子句		没有 GROUPBY 子句 时, HAVING 子句可以并入 WHERE 子句 有条件支持	没有 GROUPBY 子句或聚集函数时, HAVING 子句可以并入 WHERE 子句 有条件支持
去除表达式中冗余的括号	((a AND b)AND (c AND d))	a AND b AND c AND d 支持	支持
常量传递	col_1 = col_2 AND col_2 = 3	(col_1=3) AND (col_2=3) 支持	支持
消除死码	(0 > 1 OR a = 5)	a = 5 支持	`test`.`t1`.`a` = 5 支持
表达式计算	col_1 = 1 + 2	col_1 = 3 支持	`test`.`t2`.`col_1` = (1 + 2) 不支持
等式变换	-a = 3	(- a) = 3 不支持	-(`test`.`t2`.`a`) = 3 不支持
不等式变换	a > 10 AND b = 6 AND a > 2	(a > 10) AND (a > 2) AND (b = 2) 不支持	(`test`.`t2`.`a` > 10) and (`test`.`t2`.`a` > 2) and (`test`.`t2`.`b` = 6) 不支持
谓词传递闭包	a>b AND b>2	(a>b) AND (b>2) 不支持	(`test`.`t1`.`a` > `test`.`t2`.`b`) and (`test`.`t2`.`b` > 2) 不支持
合取项只要有一个为假, 即整个表达式为假	(0 > 1 AND s1 = 5)	False 支持	Impossible WHERE where 0 支持
AND 操作符是可交换的	col_2+col_1=100 AND col_2>1 AND col_1>2	(col_2 > 1) AND (col_1 > 2) AND ((col_2 + col_1) = 100) 支持 (不支持条件排序)	支持 [⊖] (支持条件排序) 等式优先进移

17.1.5 外连接消除

从连接类型的语义看, PostgreSQL 和 MySQL 对各种连接类型的支持都比较好, 具体如表 17-5 所示。在对外连接的优化技术上, PostgreSQL 在全外连接、半连接、反半连接的优化上较 MySQL 强, 具体如表 17-6 所示。

表 17-5 PostgreSQL 和 MySQL 连接类型对照表

连接类型	PostgreSQL 连接语法格式	MySQL 连接语法格式
内连接	[INNER CROSS] JOIN, 支持	[INNER CROSS] JOIN, 支持
左外连接	LEFT [OUTER] JOIN, 支持	LEFT [OUTER] JOIN, 支持

⊖ 原式 : a1+a2>10 AND b2=2 AND a2=3
变为 : (`test`.`t2`.`a2` = 3) and (`test`.`t2`.`b2` = 2) and ((`test`.`t1`.`a1` + 3) > 10)

(续)

连接类型	PostgreSQL 连接语法格式	MySQL 连接语法格式
右外连接	RIGHT [OUTER] JOIN, 支持	RIGHT [OUTER] JOIN, 支持
全外连接	FULL [OUTER] JOIN, 支持	FULL [OUTER] JOIN, 支持 [⊖]
半连接	EXISTS, 支持	EXISTS, 支持
反半连接	NOT EXISTS, 支持	NOT EXISTS, 支持

表 17-6 PostgreSQL 和 MySQL 连接类型的优化支持情况对照表

连接类型	PostgreSQL	MySQL
内连接	满足连接交换律, 可互换表位置, 优化点在于多表连接时对表的连接次序的选择	多表连接的次序由表的数据量决定, 即使是内连接, 也不可互换表的位置
左外连接	内表的条件列满足空值拒绝, 可优化为内连接	同 PostgreSQL
右外连接	转为左外连接, 再进行优化	同 PostgreSQL
全外连接	外表的条件列满足空值拒绝, 向左外连接转化 内表的条件列满足空值拒绝, 向右外连接转化 (右外连接又转化为左外连接的形式) 外表、内表的条件列都满足空值拒绝, 向内连接转化	不支持优化
半连接	EXISTS, 支持优化 (如用 Hash Join 优化)	EXISTS, 不支持优化
反半连接	NOT EXISTS, 支持优化 (如用 Hash Anti Join 优化)	NOT EXISTS, 不支持优化

由表 17-6 可知, 对于半连接和反半连接, 这两种连接类型的语义符合关系代数中的半连接和反半连接的定义, 在形式上看这两种连接类型的 SQL 语句属于子查询。所以, 优化的含义是对形式上的子查询而言的, 如果能消除子查询, 则意味着进行了优化。而优化后得到的查询执行计划, 可能在连接方式上标明 Hash Semi Join 类似内容, 突出半连接 (Semi) 这样的内容。PostgreSQL 对 EXISTS 支持上拉优化; MySQL 对 EXISTS 不支持上拉优化, 但对 EXISTS 子查询, MySQL 提供了 “EXISTS Strategy” 方式对其进行优化, 本书称之为 “半优化”, 具体参见 14.1.2 节。

17.1.6 嵌套连接消除

PostgreSQL 和 MySQL 都支持嵌套连接消除技术。
创建表结构如下:

```
CREATE TABLE B (b1 INT, b2 VARCHAR(9));
CREATE TABLE A (a1 INT, a2 VARCHAR(9));
CREATE TABLE C (c1 INT, c2 VARCHAR(9));
```

PostgreSQL 对嵌套连接的查询, 查询执行计划如下:

```
test=# EXPLAIN SELECT * FROM A LEFT JOIN (B LEFT JOIN C ON B.b1=C.c1) ON A.a1=B.b1 WHERE A.a1<3 AND B.b1>1
AND C.c1>2;
QUERY PLAN
```

⊖ MySQL 支持 SELECT * FROM t1 FULL JOIN t2 格式, 但 PostgreSQL 不支持这种格式。

```

Hash Join (cost=59.17..75.54 rows=41 width=132)
  Hash Cond: (c.c1 = b.b1) /* A 和 B 进行了 Hash Join 连接的结果再与 C 进行连接 */
  -> Bitmap Heap Scan on c (cost=7.09..21.68 rows=367 width=44)
    Recheck Cond: (c1 > 2)
    -> Bitmap Index Scan on c_c1_key (cost=0.00..7.00 rows=367 width=0)
      Index Cond: (c1 > 2)
  -> Hash (cost=50.55..50.55 rows=122 width=88)
    -> Hash Join (cost=33.36..50.55 rows=122 width=88) /* 先对 A 和 B 进行 Hash Join 连接 */
      Hash Cond: (a.a1 = b.b1)
      -> Bitmap Heap Scan on a (cost=7.09..21.68 rows=367 width=44)
        Recheck Cond: (a1 < 3)
        -> Bitmap Index Scan on a_a1_key (cost=0.00..7.00 rows=367 width=0)
          Index Cond: (a1 < 3)
      -> Hash (cost=21.68..21.68 rows=367 width=44)
        -> Bitmap Heap Scan on b (cost=7.09..21.68 rows=367 width=44)
          Recheck Cond: (b1 > 1)
          -> Bitmap Index Scan on b_b1_key (cost=0.00..7.00 rows=367 width=0)
            Index Cond: (b1 > 1)

```

(18 行记录)

嵌套的左外连接，按语义，B 和 C 先进行左外连接，结果再与 A 进行左外连接。PostgreSQL 的查询执行计划，消除了嵌套连接和左外连接，先对 A 和 B 进行了 Hash Join 连接，结果再与 C 进行 Hash Join 连接。另外，从查询执行计划上看，PostgreSQL 对条件的逻辑推导还不够（MySQL 能够推导出 “(‘c1’ < 3) and (‘c1’ > 1) and (‘c1’ > 2)” 这样的条件来）。

MySQL 对嵌套连接的查询，查询执行计划如下：

```
mysql> EXPLAIN EXTENDED SELECT * FROM A LEFT JOIN (B LEFT JOIN C ON B.b1=C.c1) ON A.a1=B.b1 WHERE A.a1<3 AND B.b1>1 AND C.c1>2;
```

id	select_type	table	type	Extra
1	SIMPLE	C	ALL	Using where
1	SIMPLE	B	ALL	Using where; Using join buffer (Block Nested Loop)
1	SIMPLE	A	ALL	Using where; Using join buffer (Block Nested Loop)

被查询优化器处理后的语句如下：

```

/* select#1 */ select `test`.`a`.`a1` AS `a1`,
  `test`.`a`.`a2` AS `a2`,
  `test`.`b`.`b1` AS `b1`,
  `test`.`b`.`b2` AS `b2`,
  `test`.`c`.`c1` AS `c1`,
  `test`.`c`.`c2` AS `c2`
from `test`.`a` join `test`.`b` join `test`.`c`
where ((`test`.`b`.`b1` = `test`.`c`.`c1`) and
  (`test`.`a`.`a1` = `test`.`c`.`c1`) and

```

```

('test'.c.c1 < 3) and /* 条件优化的结果, c1 相关的这 3 个条件, 可以进一步优化 */
('test'.c.c1 > 1) and
('test'.c.c1 > 2))

```

MySQL 的查询执行计划, 消除了嵌套连接和左外连接, 先对 C 和 B 进行了 join 连接, 结果再与 A 进行 join 连接。另外, 虽然 MySQL 能够推导出 “(c1 < 3) and (c1 > 1) and (c1 > 2)” 这样的条件来, 但是没有再进一步推导。如果结合 c1 的列属性为 INT 型, 则应该进一步推导出这样的条件不成立, 致使整个条件的结果值为 FALSE。

17.1.7 连接消除

PostgreSQL 和 MySQL 都不直接支持连接消除技术。但 PostgreSQL 在外连接的优化过程中, 能间接消除连接。这是外连接优化的一个附属产物。

PostgreSQL 对连接的消除 (主外键表连接), 查询执行计划如下:

```

test=# EXPLAIN SELECT A.* FROM A,B WHERE A.a1=B.b1;
          QUERY PLAN
-----
Hash Join (cost=34.75..70.88 rows=1100 width=44)
  Hash Cond: (a.a1 = b.b1)
    -> Seq Scan on a (cost=0.00..21.00 rows=1100 width=44)
    -> Hash (cost=21.00..21.00 rows=1100 width=4)
        -> Seq Scan on b (cost=0.00..21.00 rows=1100 width=4)
(5 行记录)

```

主键表 (B) 和外键表 (A) 连接, 连接条件使用主外键, 只查询外键表的列。理论上 B 可以去掉, 从而消除连接操作; 但 PostgreSQL 不支持连接消除, 依然执行了 Hash Join 操作。

MySQL 对连接的消除 (主外键表连接), 查询执行计划如下:

```

mysql> EXPLAIN EXTENDED SELECT A.* FROM A,B WHERE A.a1=B.b1;
+----+-----+-----+-----+-----+
| id | select_type | table | type | Extra |
+----+-----+-----+-----+-----+
| 1 | SIMPLE | B | index | Using index |
| 1 | SIMPLE | A | ALL | Using where; Using join buffer (Block Nested Loop) |
+----+-----+-----+-----+-----+

```

被查询优化器处理后的语句如下:

```

/* select#1 */ select `test`.`a`.`a1` AS `a1`,
      `test`.`a`.`a2` AS `a2`
from `test`.`a` join `test`.`b`
where (`test`.`a`.`a1` = `test`.`b`.`b1`)

```

MySQL 也不支持连接消除, 依然使用块嵌套连接算法执行了连接操作。

PostgreSQL 对外连接的消除 (主外键表外连接), 查询执行计划如下:

```

test=# EXPLAIN SELECT A.* FROM A LEFT JOIN B ON (a1=b1);
          QUERY PLAN
-----

```

Seq Scan on a (cost=0.00..27.70 rows=1770 width=16)
(1 行记录)

主键表（B）和外键表（A）进行左外连接，只查询外键表的列。PostgreSQL 只对 A 表进行了顺序扫描，然后执行计划结束，所以支持主外键表连接情况下消除连接。

MySQL 对外连接的消除（主外键表外连接），查询执行计划如下：

```
mysql> EXPLAIN SELECT A.* FROM A LEFT JOIN B ON (a1=b1);
+----+-----+-----+-----+-----+
| id | select_type | table | type  | Extra          |
+----+-----+-----+-----+-----+
| 1  | SIMPLE     | A     | ALL   | NULL           |
| 1  | SIMPLE     | B     | eq_ref | Using index    |
+----+-----+-----+-----+-----+
```

主键表（B）和外键表（A）进行左外连接，只查询外键表的列。MySQL 对 A、B 表进行了连接，所以不支持主外键表连接情况下消除连接。

17.1.8 语义优化

语义优化技术虽然也是对 SQL 语句的改写，但和其他的查询重写技术不同，语义优化是从上下文环境、完整性约束等进行逻辑推理判断，不是依据 SQL 形式做推断。在实际应用当中，一些工程师喜欢去掉完整性约束，把一些完整性校验交给应用程序去做，表的设计不遵循 E-R 模型和范式约束，这样既为应用程序带来额外的维护负担，又人为放弃了数据库查询优化器可以进行的语义优化功能，得不偿失。

PostgreSQL 不支持语义优化技术，MySQL 部分支持语义优化技术，具体对比如表 17-7 所示。

表 17-7 语义优化支持情况对照表

常见语义优化技术	PostgreSQL	MySQL
连接消除	不支持	不支持
唯一性使用	不支持	支持
排序优化	支持	支持（条件苛刻）
谓词引入	不支持	不支持
检测空回答集	不支持	部分支持

17.2 非 SPJ 的优化

非 SPJ 查询语句的优化是数据库查询优化器重点实现的技术，在不同的数据库中，都实现过非 SPJ 查询语句的优化，PostgreSQL 和 MySQL 的查询优化器也不例外。对非 SPJ 查询语句的优化，PostgreSQL 和 MySQL 的查询优化器在优化程度、方式等方面不尽相同。

1. 聚集函数的优化

聚集函数在索引键上使用时，可以通过索引进行优化。PostgreSQL 和 MySQL 的查询优化器对聚集函数利用索引的优化支持程度如表 17-8 所示。

表 17-8 聚集函数利用索引优化对照表

聚集函数	PostgreSQL	MySQL
count	不支持	支持
MIN/MAX	支持	支持
sum	不支持	支持
avg	不支持	支持

2. GROUPBY 优化

PostgreSQL 不能在 GROUPBY 中直接利用索引进行优化，但支持在 ORDERBY 中用索引对 ORDERBY 进行优化，而 GROUPBY 操作不能利用 ORDERBY 的成果。但 MySQL 对于这些支持得都很好，具体比较如表 17-9 所示。

表 17-9 GROUPBY 优化对照表

GROUPBY 优化技术		PostgreSQL	MySQL
分组转换技术		不支持	不支持
利用索引优化	分组列是索引列	不支持	支持
	分组、排序列是同一索引列	支持	支持

3. ORDERBY 优化

对于单纯的排序操作，PostgreSQL 和 MySQL 都能很好地利用索引，进行排序操作的优化，具体比较如表 17-10 所示。

4. DISTICT 优化

对于 DISTICT 操作，MySQL 能很好地利用索引，进行 DISTICT 优化；PostgreSQL 却不能充分利用索引进行优化，具体比较如表 17-11 所示。

表 17-10 ORDERBY 优化对照表

ORDERBY 优化技术	PostgreSQL	MySQL
排序消除（多数利用索引）	支持	支持
排序下推	支持	不支持

表 17-11 DISTICT 优化对照表

DISTICT 优化技术	PostgreSQL	MySQL
DISTICT 消除	不支持	支持
DISTICT 推入	不支持	支持

5. 集合操作优化

对 PostgreSQL 来说，集合操作优先于 SPJ 和带有 GROUPBY 等的非 SPJ，当处理条件中的谓词时，不考虑与集合操作的关联关系，所以也没有机会做“OR 重写并集规则”。如果能够实现“OR 重写并集规则”技术，则需要计算改写为并集操作的 SQL 的花费（需要重新计算改写后的 SQL 花费），与改写前的原 SQL 花费做比较。重新计算改写后的花费这种操作的耗费比较大，是否值得，需要评估。

MySQL 在处理集合操作时，与 PostgreSQL 的处理方式类似，集合操作优先于其他类型的操作，所以优缺点等同于 PostgreSQL。

6. 其他优化

其他一些特殊情况，有些是可以优化的，PostgreSQL 和 MySQL 的支持程度不同，参见如下 4 种情况。

首先，创建表，结构如下：

```
CREATE TABLE t_o1 (a1 INT UNIQUE, b1 INT);
INSERT INTO t_o1 VALUES (1, 1);
INSERT INTO t_o1 VALUES (2, 2);
```

语句一 当 LIMIT 子句限定元组数为 1，即只有一条元组，可对 DISTINCT 操作进行

优化。

```
SELECT DISTINCT * FROM t_o1 WHERE b1=2 LIMIT 1;
```

语句二 DISTINCT 与聚集函数语义都能保证唯一，可去掉 DISTINCT。

```
ELECT DISTINCT MAX(b1) FROM t_o1;
```

语句三 GROUPBY 可依据唯一索引列进行优化，DISTINCT 也可以依据唯一索引列优化。

```
SELECT DISTINCT a1 FROM t_o1 GROUP BY a1;
```

语句四 只有一个表，表的查询列是排序列，且这样的列上有索引，则 DISTINCT 可以被 ORDERBY 消除。

```
create index t_o1_i on t_o1(b1);
SELECT DISTINCT b1 FROM t_o1 ORDER By b1;
```

对于语句一，PostgreSQL 不支持优化，看下面的查询执行计划：

```
test=# EXPLAIN SELECT DISTINCT * FROM t_o1 WHERE b1=2 LIMIT 1;
```

QUERY PLAN

```
Limit (cost=1.06..1.07 rows=1 width=8)// 后执行 LIMIT
-> Unique (cost=1.06..1.07 rows=1 width=8)// 先执行 DISTINCT
    -> Sort (cost=1.06..1.06 rows=1 width=8)
        Sort Key: a1 // 去重操作要求排序，尽量利用索引排序
        -> Seq Scan on t_o1 (cost=0.00..1.05 rows=1 width=8)
            Filter: (b1 = 2)
```

(6 rows)

PostgreSQL 的查询执行计划没有去掉代表 DISTINCT 操作的 Unique 结点。

对于语句一，MySQL 不支持优化，查询执行计划如下：

```
mysql> EXPLAIN EXTENDED SELECT DISTINCT * FROM t_o1 WHERE b1=2 LIMIT 1;
```

id	select_type	table	type	Extra
1	SIMPLE	t_o1	ref	Using temporary

被查询优化器处理后的语句如下：

```
/* select#1 */ select distinct `test`.`t_o1`.`a1` AS `a1`,
`test`.`t_o1`.`b1` AS `b1`
from `test`.`t_o1`
where (`test`.`t_o1`.`b1` = 2)
limit 1
```

MySQL 的查询执行计划使用了临时表（Using temporary）做排序操作，被查询优化器处

理后的语句表明，DISTINCT 和 LIMIT 操作都被执行，所以 MySQL 不支持语句一的优化。

对于语句二，PostgreSQL 不支持优化，查询执行计划如下：

```
test=# EXPLAIN SELECT DISTINCT MAX(b1) FROM t_o1;
               QUERY PLAN
-----
Unique (cost=36.77..36.78 rows=1 width=4)
  -> Sort (cost=36.77..36.78 rows=1 width=4)
        Sort Key: (max(b1))
        -> Aggregate (cost=36.75..36.76 rows=1 width=4)
              -> Seq Scan on t_o1 (cost=0.00..31.40 rows=2140 width=4)
(5 rows)
```

PostgreSQL 先执行了聚集操作，然后执行排序，再然后进行 Unique 去重，所以 DISTINCT 与聚集函数语义都能保证唯一，可去掉 DISTINCT 操作。但语句二，去重操作和聚集操作都得到执行，PostgreSQL 不支持优化。

对于语句二，MySQL 支持优化，查询执行计划如下：

```
mysql> EXPLAIN EXTENDED SELECT DISTINCT MAX(b1) FROM t_o1;
+-----+-----+-----+-----+-----+
| id | select_type | table | type | Extra |
+-----+-----+-----+-----+-----+
| 1 | SIMPLE | NULL | NULL | Select tables optimized away |
+-----+-----+-----+-----+-----+
```

MySQL 查询计划中 Extra 列的 Select tables optimized away 值表明，MySQL 对情况二的语句进行了优化。如果不支持优化则 Extra 列的值可能为 Using temporary。

对于语句三，PostgreSQL 不支持优化，查询执行计划如下：

```
test=# EXPLAIN SELECT DISTINCT a1 FROM t_o1 GROUP BY a1;
               QUERY PLAN
-----
Unique (cost=0.00..91.05 rows=2140 width=4)
  -> Group (cost=0.00..85.70 rows=2140 width=4)
        -> Index Scan using t_o1_a1_key on t_o1 (cost=0.00..80.35 rows=2140 width=4)
(3 rows)
```

PostgreSQL 在索引的基础上执行了分组，然后又执行了去重操作，没有把 DISTINCT 和 GROUPBY 操作消除掉。

对于语句三，MySQL 支持优化，查询执行计划如下：

```
mysql> EXPLAIN SELECT DISTINCT * FROM t_o1 GROUP BY a1;
+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | key | Extra |
+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | t_o1 | index | a1 | NULL |
+-----+-----+-----+-----+-----+-----+
```

MySQL 利用索引完成分组和去重的优化。

对于语句四，PostgreSQL 不支持优化，查询执行计划如下：

```
test=# EXPLAIN SELECT DISTINCT b1 FROM t_o1 ORDER By b1;
          QUERY PLAN
-----
Unique  (cost=1.08..1.10 rows=4 width=4)
-> Sort (cost=1.08..1.09 rows=4 width=4)
    Sort Key: b1
-> Seq Scan on t_o1 (cost=0.00..1.04 rows=4 width=4)
(4 rows)
```

PostgreSQL 先排序后执行去重 (Unique) 操作，不支持优化。

对于语句四，MySQL 支持优化，查询执行计划如下：

```
mysql> EXPLAIN EXTENDED SELECT DISTINCT b1 FROM t_o1 ORDER By b1;
+-----+
| id | select_type | table | type | Extra |
+-----+
| 1 | SIMPLE | t_o1 | index | Using index |
+-----+
```

MySQL 利用索引完成优化。

17.3 本章小结

本章从逻辑查询优化的角度，对比、总结了 PostgreSQL 和 MySQL 对各种逻辑优化技术的支持情况。

对于子查询的优化，PostgreSQL 和 MySQL 各有所长；对于等价谓词重写、条件的处理，MySQL 略胜 PostgreSQL 一筹，在各种连接消除方面，二者基本相当，都支持外连接消除和嵌套连接消除。在索引和约束的利用方面，尤其是语义优化和非 SPJ 的优化，MySQL 显得技高一筹；对于索引和约束以及条件化简的充分利用，使得 MySQL 能及早把计算和推理的工作在查询计划生成的过程中完成，从而生成更为高效的查询执行计划。

整体上，MySQL 查询优化器支持的逻辑优化点比 PostgreSQL 多，MySQL 查询优化器逻辑查询优化部分灵光闪烁，让读之者爱不释手，但这不代表查询优化器的效率高于 PostgreSQL。查询优化器的效率高低需要在现实中根据实际场景通过测试来评估。

另外，本章的一些对比没有给出示例，也没有更为细致地给出一些细节的比较结果，详细内容需要参见第二篇、第三篇相应章节。所幸第二篇、第三篇内容不同，但目录结构、组织方式和命名方式均相同。所以阅读本章请参阅第二篇、第三篇相应章节（ORDERBY 子句内容的细节，本章没有完全罗列总结，需要读者自己根据前文进行对比才更有收获）。

PostgreSQL 和 MySQL 的物理查询优化技术

物理查询，本质上是把逻辑操作符转换为物理操作符的过程。物理查询优化是在这个过程中，对一些步骤和过程点进行优化。逻辑操作要使用元组，物理操作对应的就需要进行单表的扫描；逻辑操作要进行投影和选择，物理操作则要进行列的解析和表的过滤（过滤器）操作；逻辑操作要进行连接，物理操作则要进行两表连接算法。当一个查询中的表被依次扫描、过滤或连接时，按什么次序去执行这些操作也可以优化，这是多表连接的顺序选择优化（这只是物理优化方式，没有对应的逻辑优化操作）；在多种多表连接方式被查询优化器求解得到的时候，代价估算模型成为查询优化器评估每一个候选计划的依据。

PostgreSQL 和 MySQL 的物理查询优化有相似之处，也有不同之处，具体异同，参见本章以下内容。

18.1 查询代价估算模型比较

PostgreSQL 和 MySQL 都遵从典型的代价估算模型，总花费 = IO 花费 + CPU 花费，在原理上保持一致。总花费由各个操作的花费加和组成，即各个操作也有 IO 花费和 CPU 花费。

PostgreSQL 的框架结构清晰，每一种操作都有清晰的步骤与之对应，因此从函数的角度看，代价计算模型对于每一种操作来说，分解得都特别清晰，如单表顺序扫描（`cost_seq` 函数）与索引扫描（`cost_index` 函数）代价估算、两表不同的连接算法（`final_cost_mergejoin` 函数、`final_cost_nestloop` 函数、`final_cost_hashjoin` 函数）的代价估算等，都有单独的函数与之对应。也就是说，因为 PostgreSQL 从整体结构上分类清晰，所以容易理解和查找。

MySQL 的代价估算与 PostgreSQL 相比，显得有些凌乱。MySQL 代价估算主要集中在对表的访问方式的比较过程中，计算全表扫描（即顺序扫描）、各种基于索引的扫描方式时，才进行代价估算。MySQL 因有众多的存储引擎，一些计算工作需要调用存储引擎才能完成。即存储引擎可以根据自己的特点实现合适的计算方法，但总的原理是不变的。另外，

MySQL 引入了内存花费、远程花费的概念，前者是应对本地内存申请释放等耗时较大时的花费计算的，后者是分布式计算的代价估算模型需要的。

18.2 单表扫描算法

无论是 PostgreSQL 还是 MySQL，其 IO 花费主要都是单表扫描的花费，尽管数据库都采用了数据缓冲区技术，有效缓解了 IO 与 CPU 速度不匹配的问题，但 IO 花费依旧不可避免（PostgreSQL 和 MySQL 都是非内存型关系数据库）。

PostgreSQL 采取了以下 3 种单表扫描方式。

- 顺序扫描：相当于 MySQL 的全表扫描。
- 索引扫描：分为普通索引扫描（索引文件和数据文件都需要读取）和只读索引扫描（相当于 MySQL 的覆盖索引扫描，只读取索引文件）。
- Tid 扫描：类似索引扫描能快速定位元组，只不过通过元组的隐含列（隐含列是 PostgreSQL 为元组自动添加的列）标识 ctid 直接计算元组位置，然后读取相应位置的元组。

MySQL 也支持全表扫描和索引扫描。索引扫描被细分为几种类型（JT_EQ_REF、JT_REF、JT_REF_OR_NULL、JT_INDEX 等），这是 MySQL 的一个特色之处。细分通过索引获取数据的方式，使得读取元组有了差异（差异主要体现在因元组个数的不同而导致的读取 IO 的花费不同），同时还使得多表连接的顺序得到确定，有利于查询优化器识别连接时的情况（因为 MySQL 的多表连接算法需要依据表的元组个数决定表的连接次序）。

18.3 索引

所有的数据库都提供索引技术，而且索引是最有效的提高查询优化的方式。索引支持的程度、索引类型等各个数据库都不尽相同，这和查询优化这个主题不直接相关，在此不做比较。

索引对查询各个子句的影响，在 PostgreSQL 和 MySQL 的相应章节进行了描述，其对比也不再赘述。

索引适用于选择率较低的情况，因为索引会导致随机读，大量使用索引将导致大量的随机读，这会降低查询效率。

PostgreSQL 和 MySQL 对于索引的利用能力有所不同，相比之下，MySQL 利用索引的能力更强。MySQL 不仅能利用前缀索引，甚至当查询语句中只包括索引键的非前缀部分的关键字时，相应的索引也能被利用。

索引对 PostgreSQL 和 MySQL 的影响对比如表 18-1 所示。

表 18-1 利用索引进行优化的情况对照表

表达式原型	优化支持情况	
	PostgreSQL	MySQL
利用索引列进行 COUNT(*) 优化 (语句一, 无条件子句, 直接利用索引)	不支持	支持
利用索引进行 COUNT(*) 优化 (语句二, 索引列出现在条件子句中)	不支持	支持
普通索引对 MIN/MAX 的优化 (语句三)	支持	支持
普通索引对 MIN/MAX 在非前缀索引键上的优化 (语句四)	支持	支持
主键索引对排序的优化 (语句五)	支持	支持
普通索引对排序的优化 (语句六)	支持	不支持
普通索引对分组操作的优化 (语句七)	支持	支持
索引对于 DISTINCT 操作的优化 (语句八)	不支持	支持
用索引键的一部分完成对表的查询 (语句九)	支持	支持
多个索引键存在, 非前缀索引和主键索引列混合做条件, 利用主键索引进行优化 (语句十)	支持	支持
非前缀索引和非索引列混合做条件 (语句十一)	支持	不支持

索引对 PostgreSQL 和 MySQL 的影响对比我们通过表 18-1 已经初步了解了, 为了加深读者的印象, 我们来看一下表 18-1 中提到的各条语句的具体代码, 如表 18-2 所示。为了帮助大家更好地理解表 18-2 中的语句, 先创建下面这些对象:

```
CREATE TABLE tkey (key_pk INT, key_part1 INT, key_part2 INT, nokey INT, PRIMARY KEY(key_pk));
CREATE INDEX tkey_multi_k ON tkey (key_part1, key_part2);
INSERT INTO tkey VALUES (1,1,1,1);
INSERT INTO tkey VALUES (2,2,1,1);
INSERT INTO tkey VALUES (3,2,2,1);
INSERT INTO tkey VALUES (4,3,1,1);
INSERT INTO tkey VALUES (5,3,2,1);
INSERT INTO tkey VALUES (6,3,3,1);
INSERT INTO tkey VALUES (7,4,1,1);
INSERT INTO tkey VALUES (8,4,2,1);
INSERT INTO tkey VALUES (9,4,3,NULL);
```

表 18-2 索引利用情况对照表对应的 SQL 语句一览表

语句编号	SQL 语句
语句一	SELECT COUNT(*) FROM tkey;
语句二	SELECT COUNT(*) FROM tkey WHERE key_part1=2 AND key_part2=1;
语句三	SELECT MIN(key_part1),MAX(key_part1) FROM tkey;
语句四	SELECT MAX(key_part2) FROM tkey WHERE key_part1=2;
语句五	SELECT * FROM tkey ORDER BY key_pk LIMIT 2;
语句六	SELECT * FROM tkey ORDER BY key_part1, key_part2 LIMIT 2;
语句七	SELECT key_part2 FROM tkey GROUP BY key_part1;
语句八	SELECT DISTINCT key_pk FROM tkey;
语句九	SELECT key_part1, key_part2 FROM tkey WHERE key_part1=1;
语句十	SELECT MIN(key_part2),MAX(key_part2) FROM tkey;

(续)

语句编号	SQL 语句
语句十一	SELECT * FROM tkey WHERE key_part2=2 and nokey>2; SELECT * FROM tkey WHERE key_part2=2;

下面针对表 18-2 中的各条语句进行简单分析。
对于语句一，PostgreSQL 不能利用索引进全表计数，看下面的查询执行计划：

```
test=# EXPLAIN SELECT COUNT(*) FROM tkey;
          QUERY PLAN
-----
Aggregate (cost=32.13..32.13 rows=1 width=0)
-> Seq Scan on tkey (cost=0.00..27.70 rows=1770 width=0)
(2 行记录)
```

PostgreSQL 的查询执行计划先执行顺序扫描，再执行聚集操作完成计数。
对于语句一，MySQL 能利用索引进全表计数，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT COUNT(*) FROM tkey;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | type | key          | Extra          |
+----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | tkey  | index | tkey_multi_k | Using index    |
+----+-----+-----+-----+-----+-----+
```

MySQL 的查询执行计划利用索引进行全表计数。
对于语句二，PostgreSQL 不能利用索引进全表计数，看下面的查询执行计划：

```
test=# EXPLAIN SELECT COUNT(*) FROM tkey WHERE key_part1=2 AND key_part2=1;
          QUERY PLAN
-----
Aggregate (cost=8.27..8.28 rows=1 width=0)
-> Index Only Scan using tkey_multi_k on tkey (cost=0.00..8.27 rows=1 width=0)
    Index Cond: ((key_part1 = 2) AND (key_part2 = 1))
(3 行记录)
```

PostgreSQL 的查询执行计划，利用索引进进行扫描，然后执行了聚集操作完成计数。
对于语句二，MySQL 能利用索引进全表计数，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT COUNT(*) FROM tkey WHERE key_part1=2 AND key_part2=1;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | type | key          | Extra          |
+----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | tkey  | ref  | tkey_multi_k | Using index    |
+----+-----+-----+-----+-----+-----+
```

MySQL 的查询执行计划的 Extra 列的值为 Using index，表明 MySQL 能利用索引进行计数（索引列做条件子句）。

对于语句三，PostgreSQL 支持用只读索引扫描完成在索引键上的聚集操作，看下面的

查询执行计划：

```
test=# EXPLAIN SELECT MIN(key_part1),MAX(key_part1) FROM tkey;
                                QUERY PLAN
-----
Result (cost=0.09..0.10 rows=1 width=0)
  InitPlan 1 (returns $0)
    -> Limit (cost=0.00..0.04 rows=1 width=4)
      -> Index Only Scan using tkey_multi_k on tkey (cost=0.00..79.07 rows=1761 width=4)
          Index Cond: (key_part1 IS NOT NULL)
  InitPlan 2 (returns $1)
    -> Limit (cost=0.00..0.04 rows=1 width=4)
      -> Index Only Scan Backward using tkey_multi_k on tkey (cost= 0.00..79.07 rows=1761 width=4)
          Index Cond: (key_part1 IS NOT NULL)
```

(9 行记录)

PostgreSQL 的查询执行计划，利用两次索引只读扫描（Index Only Scan），完成 MIN、MAX 的求值。

对于语句三，MySQL 支持用索引扫描完成在索引键上的聚集操作，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT MIN(key_part1),MAX(key_part1) FROM tkey;
+-----+-----+-----+-----+-----+
| id | select_type | table | key | Extra |
+-----+-----+-----+-----+-----+
| 1 | SIMPLE | NULL | NULL | Select tables optimized away |
+-----+-----+-----+-----+-----+
```

MySQL 的查询执行计划的 Extra 列值 Select tables optimized away 表明，MySQL 能用索引完成 MIN、MAX 的求值。

对于语句四，PostgreSQL 支持用只读索引扫描完成在非前缀索引键上的聚集操作，看下面的查询执行计划：

```
test=# EXPLAIN SELECT MAX(key_part2) FROM tkey WHERE key_part1=2;
                                QUERY PLAN
-----
Result (cost=3.60..3.61 rows=1 width=0)
  InitPlan 1 (returns $0)
    -> Limit (cost=0.00..3.60 rows=1 width=4)
      -> Index Only Scan Backward using tkey_multi_k on tkey (cost= 0.00..32.43 rows=9 width=4)
          Index Cond: ((key_part1 = 2) AND (key_part2 IS NOT NULL))
```

(5 行记录)

PostgreSQL 的查询执行计划中，用只读索引扫描完成在非前缀索引键上的 MAX 聚集操作，没有出现额外的聚集操作结点。

对于语句四，MySQL 支持用索引扫描完成在索引键上的聚集操作，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT MAX(key_part2) FROM tkey WHERE key_part1=2;
+-----+
| id | select_type | table | key | Extra |
+-----+
| 1 | SIMPLE | NULL | NULL | Select tables optimized away |
+-----+
```

MySQL 的查询执行计划的 Extra 列值 Select tables optimized away 表明，MySQL 能利用索引完成在非前缀索引键上的 MAX 聚集操作。

对于语句五，PostgreSQL 支持用主键索引优化排序操作，看下面的查询执行计划：

```
test=# EXPLAIN SELECT * FROM tkey ORDER BY key_pk LIMIT 2;
               QUERY PLAN
-----
Limit (cost=0.00..0.08 rows=2 width=16)
-> Index Scan using tkey_pkey on tkey (cost=0.00..74.80 rows=1770 width=16)
(2 行记录)
```

PostgreSQL 的查询执行计划中，使用索引扫描替代了排序操作（sort 操作没有出现在查询执行计划中）。

对于语句五，MySQL 主键索引替代排序，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT * FROM tkey ORDER BY key_pk LIMIT 2;
+-----+
| id | select_type | table | type | key | Extra |
+-----+
| 1 | SIMPLE | tkey | index | PRIMARY | NULL |
+-----+
```

MySQL 的查询执行计划中，type 列的值为 index，Extra 列的值为 NULL，没有出现类似 filesort 或 temporary 字样的内容，表明 MySQL 支持用主键索引替代排序。

对于语句六，PostgreSQL 支持用普通索引优化排序操作，看下面的查询执行计划：

```
test=# EXPLAIN SELECT * FROM tkey ORDER BY key_part1, key_part2 LIMIT 2;
               QUERY PLAN
-----
Limit (cost=0.00..0.08 rows=2 width=16)
-> Index Scan using tkey_multi_k on tkey (cost=0.00..74.80 rows=1770 width=16)
(2 行记录)
```

PostgreSQL 的查询执行计划中，使用索引扫描替代了排序操作（sort 操作没有出现在查询执行计划中）。

对于语句六，MySQL 不支持用普通索引优化排序操作，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT * FROM tkey ORDER BY key_part1, key_part2 LIMIT 2;
+-----+
| id | select_type | table | type | key | Extra |
+-----+
| 1 | SIMPLE | tkey | ALL | NULL | Using filesort |
+-----+
```

MySQL 的查询执行计划中, Extra 列的值为 Using filesort, 表明 MySQL 不支持用普通索引(非主键索引)优化排序操作。

对于语句七, PostgreSQL 语法不支持与 MySQL 同类型操作; 对 SQL 语句进行改造后可执行, 但不支持用索引完成分组操作, 看下面的查询执行计划:

```
test=# EXPLAIN SELECT key_part1 FROM tkey GROUP BY key_part1;
QUERY PLAN
```

```
HashAggregate (cost=32.13..34.13 rows=200 width=4)
-> Seq Scan on tkey (cost=0.00..27.70 rows=1770 width=4)
(2 行记录)
```

PostgreSQL 的查询执行计划, 执行了聚集操作 (HashAggregate), 所以不支持用索引完成分组操作。

对于语句七, MySQL 支持用索引完成分组操作, 看下面的查询执行计划:

```
mysql> EXPLAIN SELECT key_part2 FROM tkey GROUP BY key_part1;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | type | key | Extra |
+----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | tkey | index | tkey_multi_k | Using index |
+----+-----+-----+-----+-----+-----+
```

MySQL 的查询执行计划中, Extra 列的值为 Using index, 表明 MySQL 支持用索引完成分组操作。

对于语句八, PostgreSQL 不支持利用索引对 DISTINCT 优化, 看下面的查询执行计划:

```
test=# EXPLAIN SELECT DISTINCT key_pk FROM tkey;
QUERY PLAN

HashAggregate (cost=32.13..49.83 rows=1770 width=4)
-> Seq Scan on tkey (cost=0.00..27.70 rows=1770 width=4)
(2 行记录)
```

PostgreSQL 的查询执行计划中, 使用了聚集操作完成 DISTINCT 操作, 不支持利用索引对 DISTINCT 优化。

对于语句八, MySQL 支持用索引完成 DISTINCT 操作, 看下面的查询执行计划:

```
mysql> EXPLAIN SELECT DISTINCT key_pk FROM tkey;
+----+-----+-----+-----+-----+-----+
| id | select_type | table | type | Extra |
+----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | tkey | index | Using index |
+----+-----+-----+-----+-----+-----+
```

```
mysql> EXPLAIN SELECT DISTINCT key_part1 FROM tkey;
```

id	select_type	table	type	Extra
1	SIMPLE	tkey	index	Using index

```
mysql> EXPLAIN SELECT DISTINCT key_part2 FROM tkey;
```

id	select_type	table	type	Extra
1	SIMPLE	tkey	index	Using index; Using temporary

```
mysql> EXPLAIN SELECT DISTINCT * FROM tkey;
```

id	select_type	table	type	Extra
1	SIMPLE	tkey	ALL	NULL

MySQL 前 3 条 SQL 语句的查询执行计划中，Extra 列的值为 Using index，表明 MySQL 支持在主键的索引键上、在前缀索引键上、非前缀索引键上对 DISTINCT 优化。最后一条 SQL 语句的查询执行计划中，Extra 列的值为 NULL，表明 MySQL 在无索引可用的情况下，不对 DISTINCT 操作进行优化。

对于语句九，PostgreSQL 支持用部分索引完成对索引键的查询，看下面的查询执行计划：

```
test=# EXPLAIN SELECT key_part1, key_part2 FROM tkey WHERE key_part1=1;
QUERY PLAN
```

```
Bitmap Heap Scan on tkey (cost=4.32..14.86 rows=9 width=8)
  Recheck Cond: (key_part1 = 1)
  -> Bitmap Index Scan on tkey_multi_k (cost=0.00..4.32 rows=9 width=0)
      Index Cond: (key_part1 = 1)
(4 行记录)
```

PostgreSQL 的查询执行计划中，利用部分索引键 key_part1 进行位图索引扫描（Bitmap Index Scan），完全支持用部分索引完成对索引键的查询。

对于语句九，MySQL 支持用部分索引完成对索引键的查询，看下面的查询执行计划：

```
mysql> EXPLAIN SELECT key_part1, key_part2 FROM tkey WHERE key_part1=1;
```

id	select_type	table	type	key	Extra
1	SIMPLE	tkey	ref	tkey_multi_k	Using index

MySQL 的查询执行计划中, Extra 列的值为 Using index, 表明 MySQL 支持用部分索引键 key_part1 进行索引扫描。

对于语句十, PostgreSQL 不支持用部分索引完成对 MIN、MAX 的聚集优化, 看下面的查询执行计划:

```
test=# EXPLAIN SELECT MIN(key_part2),MAX(key_part2) FROM tkey;
               QUERY PLAN
-----
Aggregate (cost=1.14..1.15 rows=1 width=4)
-> Seq Scan on tkey (cost=0.00..1.09 rows=9 width=4)
(2 行记录)
```

PostgreSQL 的查询计划中, 使用了索引扫描, 然后使用了聚集操作, 所以 PostgreSQL 不支持用部分索引完成对 MIN、MAX 的聚集优化。

对于语句十, MySQL 支持查询语句只包括部分索引键且不使用索引前缀部分时, 用索引完成对查询语句的优化。看下面的查询执行计划:

```
mysql> EXPLAIN SELECT MIN(key_part2),MAX(key_part2) FROM tkey;
+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | key          | rows | Extra          |
+----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | tkey  | index | tkey_multi_k | 9    | Using index    |
+----+-----+-----+-----+-----+-----+-----+
1 row in set (0.03 sec)
```

MySQL 的查询执行计划中, Extra 列的值为 Using index, 表明 MySQL 支持用部分索引键 key_part2 进行索引扫描。

对于语句十一, PostgreSQL 在非前缀索引键单独使用时, 采取了顺序扫描; 但索引键和非索引列一起使用, 则使用了没有前缀的索引键。看下面的查询执行计划:

```
test=# EXPLAIN SELECT * FROM tkey WHERE key_part2=2 and nokey>2;
               QUERY PLAN
-----
Bitmap Heap Scan on tkey (cost=21.53..32.09 rows=3 width=16)
  Recheck Cond: (key_part2 = 2)
  Filter: (nokey > 2)
-> Bitmap Index Scan on tkey_multi_k (cost=0.00..21.53 rows=9 width=0)
    Index Cond: (key_part2 = 2)
(5 行记录)
test=# EXPLAIN SELECT * FROM tkey WHERE key_part2=2;
               QUERY PLAN
-----
Seq Scan on tkey (cost=0.00..32.13 rows=9 width=16)
  Filter: (key_part2 = 2)
(2 行记录)
```

PostgreSQL 第一个 SQL 语句中的 WHERE 条件，混合和索引键 `key_part2` 和非索引键 `nokey`，查询执行计划中使用位图索引进行扫描。第二个 SQL 语句中的 WHERE 条件，使用了索引键 `key_part2`，查询执行计划中使用顺序扫描完成查询。

对于语句十一，MySQL 在非前缀索引键单独使用时，采取了顺序扫描；在索引键和非索引列一起使用时，也采取了顺序扫描。看下面的查询执行计划：

```
mysql> EXPLAIN SELECT * FROM tkey WHERE key_part2=2'and nokey>2;
```

```
+-----+
| id | select_type | table | type | key  | rows | Extra      |
+-----+
| 1 | SIMPLE      | tkey  | ALL  | NULL | 9    | Using where |
+-----+
1 row in set (0.00 sec)
```

```
mysql> EXPLAIN SELECT * FROM tkey WHERE key_part2=2;
```

```
+-----+
| id | select_type | table | type | key  | rows | Extra      |
+-----+
| 1 | SIMPLE      | tkey  | ALL  | NULL | 9    | Using where |
+-----+
1 row in set (0.00 sec)
```

MySQL 第一个 SQL 语句中的 WHERE 条件，混合了索引键 `key_part2` 和非索引键 `nokey`，查询执行计划中使用顺序扫描。第二个 SQL 语句中的 WHERE 条件，使用了索引键 `key_part2`，查询执行计划中使用顺序扫描完成查询。所以，对于非前缀索引和非索引列混合做条件的情况，MySQL 不能利用索引进行优化。

18.4 两表连接算法

PostgreSQL 和 MySQL 都支持嵌套循环连接算法。但 PostgreSQL 支持基于 Hash 连接算法和嵌套归并算法，这两种算法 MySQL V5.6.10 尚不支持，但预计很快会增加类似 Hash 的连接算法。MySQL 提出一种称为 BKA 的算法，能有效解决利用索引造成大量随机读的问题。这是因为 MySQL 提供了一种称为 MRR 的技术，通过把多个读操作排列成磁盘顺序读取的序列，以减少磁头往返移动消耗（相对于 MySQL 的 MRR 技术，PostgreSQL 提供了 BitMap Index Scan 技术，其有着与 MRR 异曲同工之妙的作用，这两种技术都可以在利用索引的情况下，有效减少随机读操作）。

MySQL 的两表连接算法有一个不同于 PostgreSQL 的地方，就是 MySQL 可以为每个连接的表分配缓存。MySQL 支持的 BNL、BKA 算法都和缓存紧密相关，此缓存不是数据缓冲区，而是专门用于连接的缓存，连接算法在缓存的基础上进行了优化。

18.5 多表连接算法

在多表连接算法上，PostgreSQL 采用了动态规划算法和遗传算法两种算法，前者能得到最优的查询执行计划，比 MySQL 的贪婪算法要好，因为 MySQL 的贪婪算法得到的是局部最优的查询执行计划。PostgreSQL 的动态规划算法遍历了所有的可能，必然能从中找出最优。但是，PostgreSQL 的动态规划算法搜索的可能组合数目会急剧增长，搜索的花费耗时巨大，这是一个弊端。PostgreSQL 通过采用遗传算法解决了搜索空间膨胀的问题，但未必能得到最优的查询执行计划。

MySQL 的贪婪算法具有思路简单的优点，其主要依据表的数据量进行表的排序，认为小数据量的表连接大数据量的表会产生最优的查询执行计划，所以不会考虑诸如内连接表互换位置的连接方式的可能（PostgreSQL 比较了 A 内连接 B 和 B 内连接 A 这两种连接方式的花费，择优留用）；MySQL 通过剪枝优化技术完成对同类型处理的丢弃，如内连接时 A 连接 B 和 B 连接 A 两种方式，B 连接 A 的方式会被剪枝掉。

在多表连接的过程中，用户的连接语义是不会被破坏的，所有的连接算法都要优先遵从连接语义（如外连接的外表优先于内表），MySQL 在多表连接前进行表的排序过程中体现了这点（所以 MySQL 代码中找不出连接类型的表达），PostgreSQL 在连接过程中根据连接类型（内连接、外连接、半连接、反半连接等类型）完整保证了用户语义不被破坏。

18.6 本章小结

本章从物理查询优化的角度，对比、总结了 PostgreSQL 和 MySQL 对各项物理优化技术的支持情况。

各种数据库，在物理查询优化方面，基本上都采用了基于代价计算的优化方式，PostgreSQL 和 MySQL 也是如此，代价估算成为物理优化的核心基础。

从单表扫描到两表连接，再到多表连接，本章对 PostgreSQL 和 MySQL 进行了对比，点出了它们主要的不同之处。

第 19 章

PostgreSQL 和 MySQL 的其他异同

本章将从查询优化器的整体角度对比 PostgreSQL 和 MySQL，主要内容包括启发式规则的比较、基本概念的比较、数据结构的比较、设计思想的比较、编码规范的比较。

对于设计思想的比较，本章根据第二篇和第三篇对 PostgreSQL 和 MySQL 查询优化器源码的分析，以及本篇前两章对 PostgreSQL 和 MySQL 的逻辑优化技术和物理优化技术的实现对比，进行了总结性的比较，对 PostgreSQL 和 MySQL 查询优化器的设计思想进行总体评述。

19.1 启发式规则的使用比较

PostgreSQL 和 MySQL 都使用了大量的启发式规则，帮助提高查询优化器的优化效率，包括逻辑优化和物理优化两个方面。常见的启发式规则对比如表 19-1 所示。

表 19-1 常见的启发式规则对比表

启发式规则	PostgreSQL	MySQL
优先做选择和投影（连接条件在查询树上下推）	支持	支持
子查询的消除	支持	支持
嵌套连接的消除	支持	支持
外连接的消除	支持	支持
连接的消除	部分支持	不支持
使用等价谓词重写对条件化简	支持	支持
语义优化	不支持	支持
分组的合并	不支持	不支持
借用索引优化分组、排序、DISTINCT 等操作	支持	支持
对视图的查询变为基于表的查询	支持	支持
连接条件的下推	支持	支持
分组的下推	不支持	不支持

(续)

启发式规则	PostgreSQL	MySQL
谓词的上拉	支持	支持
用连接取代集合操作	不支持	不支持
用 UNION ALL 取代 OR 操作	不支持	不支持

19.2 综合比较

本节从查询优化器的整体上对 PostgreSQL 和 MySQL 做对比，包括基本概念的比较、数据结构的比较、设计思想的比较、编码规范的比较。

19.2.1 基本概念的比较

PostgreSQL 和 MySQL 都是关系型数据库，基本概念一致，遵从 SQL 标准。

MySQL 独特之处是细分了表的访问方式，提出了常量表、多种基于索引的数据访问方式的概念，这点和 PostgreSQL 有着明显不同。这样做的好处是能充分利用索引完成各项操作。

另外，在代码内部，MySQL 还提出 Disk-Sweep Multi-Range Read (MRR) 技术，有效缓解了利用索引而造成大量随机读的现象。这项技术与 PostgreSQL 中的 BitMap Index Map 功效相当。但 MySQL 的 MRR 技术的应用范围比 PostgreSQL 只限于索引的 BitMap Index Map 更为广泛，所有批量读磁盘的 IO 请求都可以通过 MRR 进行优化。

19.2.2 数据结构的比较

PostgreSQL 和 MySQL 都提供了大量的数据结构，用以表达查询优化过程中的对象等。

PostgreSQL 提供的数据结构有明显的阶段性：语法分析对应查询树 (Query)，查询优化阶段对应查询路径 (Path)、查询计划 (Plan)、计划信息 (PlannerInfo)，查询执行阶段对应查询执行计划结点 (PlannedStmt)。几个主要的数据结构之间的关系是并列的，没有从属关系，本阶段需要的上阶段的信息被传递到本阶段，其他则被抛弃，这样可以节约空间，将不相关的数据结构占用的空间尽早释放掉。

MySQL 提供的数据结构各个阶段都要使用，只是各个阶段的信息作为子成员在共同的数据结构上存在 (如 JOIN 对象贯穿了查询优化和查询执行两个阶段)。这样的好处是前面的信息后面可以直接利用，为进一步优化提供了可能 (如可以考虑把集合操作的优化和 SPJ 的优化结合起来，进行“用 UNION ALL 取代 OR 操作”的优化)。

其他异同参见 PostgreSQL 和 MySQL 的相关章节。

19.2.3 设计思想的比较

对 PostgreSQL 和 MySQL 的设计思想进行比较，这是一项艰难的工作，这里笔者竭尽

所能，勉强一试。

对于 PostgreSQL 来说，从设计思想的角度看源代码，结构清晰（如代码结构划分使得可读性增强）、分层实现（如逻辑优化阶段和物理优化阶段界线明显）、分类合理（如单表连接、两表连接、多表连接分解清晰）、易于理解。大量逻辑优化技术的实现，使得 SQL 的优化技术被发挥得淋漓尽致。运用动态规划算法力求得到最优的查询执行计划，追求完美。运用遗传算法很好地解决了十几个甚至更多表的多表连接的查询执行计划生成效率问题，其致力于高效解决实际问题。

对于 MySQL 来说，走得是“巧”的路子，不讲究稳扎稳打、大开大合，所以结构较为混乱，实现思路不如 PostgreSQL 清晰。但是 MySQL 在实用上下足了功夫，处处闪烁着“优化的火花”。从优化器开始，MySQL 就着手安排对子查询进行优化，包括消除冗余子句、化解特殊格式的子查询，甚至充分利用索引对子查询进行优化（如 ALL 类型子查询）。再往后，大量运用逻辑优化的技术，也使得 SQL 的优化技术发挥得淋漓尽致。在条件表达式优化中，MySQL 优化器能对表达式进行深度求解，有效地利用了结果值对执行计划进行进一步优化，这一点比 PostgreSQL 要强（PostgreSQL 在逻辑查询优化阶段止步于表达式求解，因而不能利用结果进一步对优化提出建议）。MySQL 对于完整性约束语义的运用也比较充分，可以指导优化器对 SQL 提出进一步的优化，PostgreSQL 没有开展语义优化的工作。MySQL 对完整索引列、局部索引列利用的效果要好于 PostgreSQL，大多数情况下，PostgreSQL 只能利用完整的索引列进行索引优化，弱于 MySQL 可利用部分索引进行优化的功能。另外，MySQL 对于非 SPJ 的 SQL 优化方式运用得也较多，这主要表现在利用索引完成 ORDERBY、GROUPBY、DISTINCT、LIMIT 等子句的优化上。MySQL 的物理优化手段如两表连接算法、多表连接等没有 PostgreSQL 丰富，还有改进空间。

纵观 MySQL 查询优化器的代码，能够理出一个思路，这个思路遵循着 SQL 输入、语法解析、逻辑查询优化、物理查询优化、查询执行这条路线顺序执行。只是这条路线上，相比 PostgreSQL，让人读代码时颇费周折（结构不清晰、代码可读性不强导致理解困难）。而从 PostgreSQL 则可以清晰地看出同 MySQL 一样的线路，且线路上散落的优化“珍珠”不互相参杂，颗颗夺目。

所以，设计上，MySQL 走的是实用的路子，怎么快怎么来。PostgreSQL 则走学院派的路子，理论和实现都很清晰。

19.2.4 编码规范的比较

比较代码的编码规范则是个简单轻松的事情，只要打开代码，小读任何一段，高下立现。

PostgreSQL 的代码风格明显强于 MySQL，可读性特别好，读它的代码完全就是一种享受，有着明显的美感。

MySQL 的代码却显得很晦涩，阅读困难，不便于理解。不过 MySQL 似乎认识到了这一点，新版本的可读性正迎头赶上，V5.6 版本的代码可读性比之前的早期版本有着显著改善，但路漫漫其修远兮，还需要继续努力。

19.3 本章小结

本章从不同的角度，对 PostgreSQL 和 MySQL 的查询优化器进行了比较，当然，这些内容都是笔者的个人看法，仅供参考。

对 PostgreSQL 和 MySQL 的查询优化器的实现，需要仔细研读代码，建议读者尽量多阅读几遍代码，在阅读过程中要与本书结合、与 PostgreSQL 和 MySQL 的比较结合、与 SQL 实践结合，相互印证，这样才能起到事半功倍的效果。

在跟着本书进行完这一轮完整的学习和比较后，如果读者能再次阅读本书和相关代码，再次比较体会，如此周而复始，定有收获。

附录 A

如何掌握数据库内核

掌握数据库内核，需要积年累月地艰苦努力，不断学习。其中，掌握理论，用理论指导学习和实践最为重要。除此之外，静态阅读源代码、动态调试跟踪，在实践中不断摸索比较，进行深度的思考和总结，是掌握数据库内核的好方式。

A.1 代码阅读，最佳静态的理解方式

要想理解源代码，阅读源代码必不可少。找一个好的帮手，有助于高效阅读代码。下面分别介绍使用 Source Insight 阅读 PostgreSQL 代码、使用 Eclipse 阅读 MySQL 代码的方式。

A.1.1 Source Insight

Source Insight 是阅读、编辑代码的利器，它能对 C/C++、C# 和 Java 等程序进行分析，能分析源代码、动态维护它自己的符号数据库，并自动显示有用的上下文信息。它还能显示 reference trees、class inheritance diagrams 和 call trees 等，方便用户理解代码框架结构与相互间的关系。

下面就来介绍一下如何使用 Source Insight 阅读代码。

1. 安装 Source Insight

这里我们以在 Windows 系统下安装 SourceInsight V3.5 为例。

1) 下载 SourceInsight V3.5 安装包，下载地址如下：

<http://www.sourceinsight.com/>

2) 建立目录 E:\DBCode\PostgreSQL-9.2.3，解压 postgresql-9.2.3.tar.gz 到此目录。

3) 建立目录 E:\DBCode\PostgreSQL-9.2.3\project，用于存放工程文件。

2. 配置 Source Insight

1) 启动 Source Insight，启动后的界面如图 A-1 所示。选择 View 菜单中的 Context

Window（主窗口左下位置，用于显示主窗口中光标选中对象的子函数、变量定义等的代码）和 Relation Window（主窗口右下位置，用于显示主窗口中光标选中对象的子函数、变量定义等的调用关系，可以帮助有效理解上下文关系）窗口。

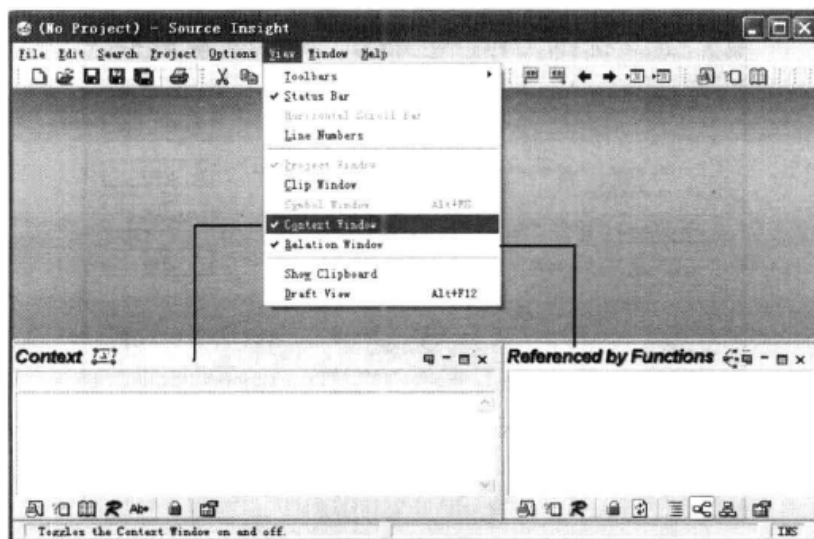


图 A-1 Source Insight 主窗口

2) 配置 Context Window 窗口的“属性”（Context 窗口最下方工具条上，单击带有“手”的形状的图标），出现配置项如图 A-2 所示。

3) 配置 Relation Window 窗口的“属性”（Referenced by Functions 窗口最下方工具条上，单击带有“手”的形状的图标），出现配置项如图 A-3 所示。注意在 View Relationship 选项组中，选择 Referenced By Functions 选项。

4) 在配置主窗口，待工程文件建立后，在主窗口，即代码显示的窗口中右击，然后在弹出的快捷菜单中选择 Line Numbers，目的是为代码显示行号，便于阅读。

5) 在主窗口的菜单 Options 中选择 Document Options 选项，配置文档窗口属性，如图 A-4 所示。这里要进行的主要配置如下：

- ❑ Document Type 选择 C Source Type。
- ❑ File filter 中填写 “*.c;*.h;*.y;*.sh;*.def;*.l”，把 PostgreSQL 中的主要文件类型包括进去，如 *.y 是要把 gram.y 这样的语

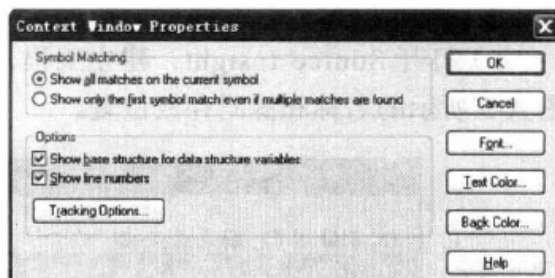


图 A-2 Context Window 窗口属性配置

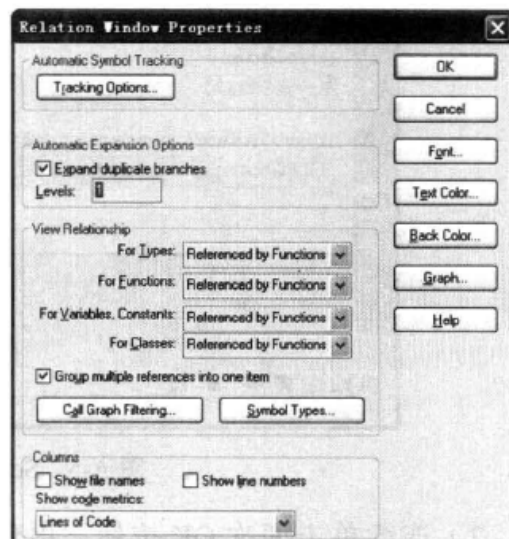


图 A-3 Relation Window 窗口属性配置

法文件包括到工程中。

- ❑ 在 Screen Fonts 中选择一个等宽的字体，大小适合自己即可。
- ❑ 在 Tab width 中填写 4 为佳。

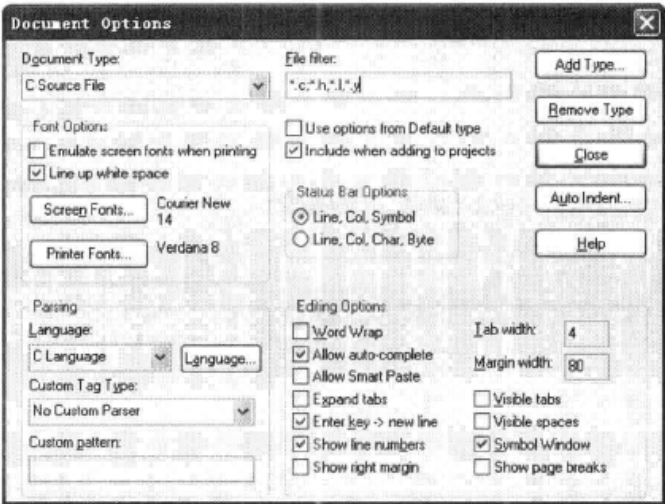


图 A-4 配置文档窗口属性

3. 建立工程

1) 打开 Source Insight，建立工程，选择 Project 菜单中的 New Project 选项，输入如图 A-5 所示的名称和源码存放位置。

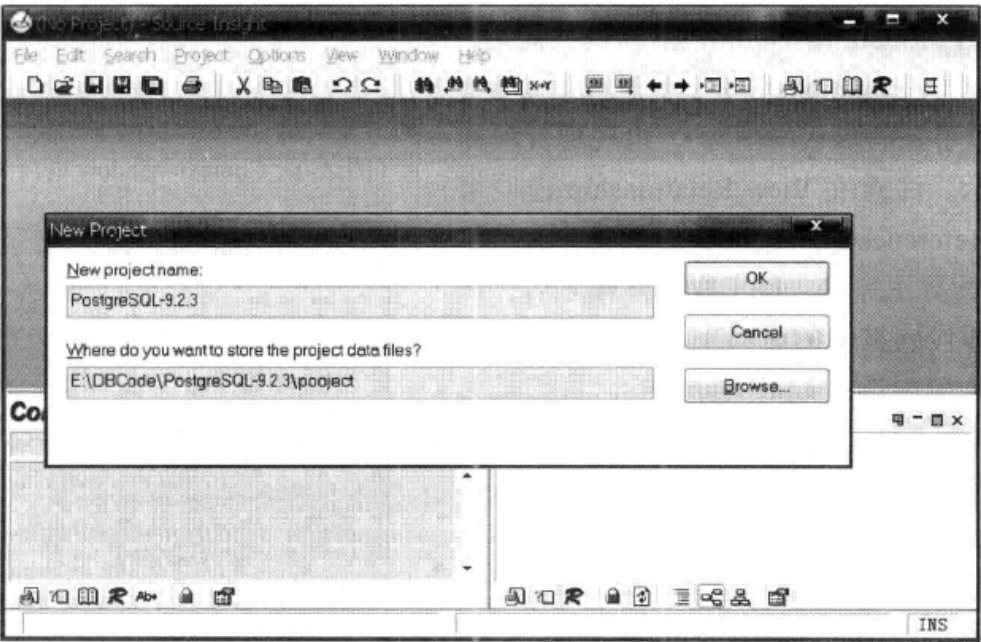


图 A-5 Source Insight 新建工程主界面

2) 连续单击两次 OK 按钮，出现如图 A-6 所示界面。选择 E:\DBCode\PostgreSQL-9.2.3\src 目录，单击 Add Tree 按钮，出现如图 A-7 所示提示框。

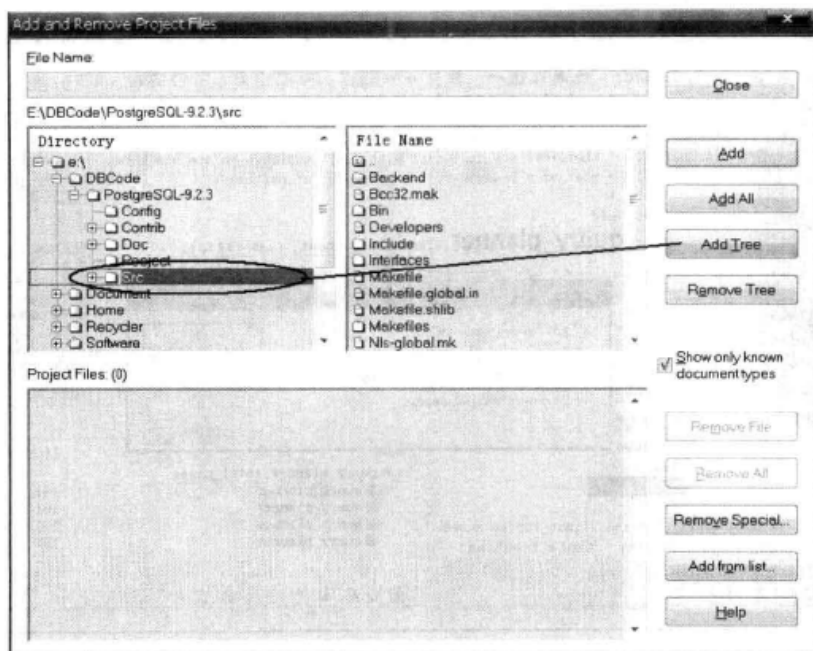


图 A-6 Source Insight 新建工程导入源码



图 A-7 Source Insight 新建工程导入源码提示框

3) Source Insight 将把选中的文件加入工程, 如图 A-8 所示。选中目录 E:\DBCode\PostgreSQL-9.2.3\src\test\regress\data (没有用处, 可以去掉, 加快 Source Insight 的响应速度), 单击 Remove Tree 按钮。

4) 在 Source Insight 的 UI 右侧的 Project Window 中输入 planmian.c, 然后按回车键, 主窗口出现 planmian.c 文件内容。

5) 在 Project 菜单中选择 Synchronize Files 选项, 弹出如图 A-9 所示对话框。在此可以建立文件的索引。

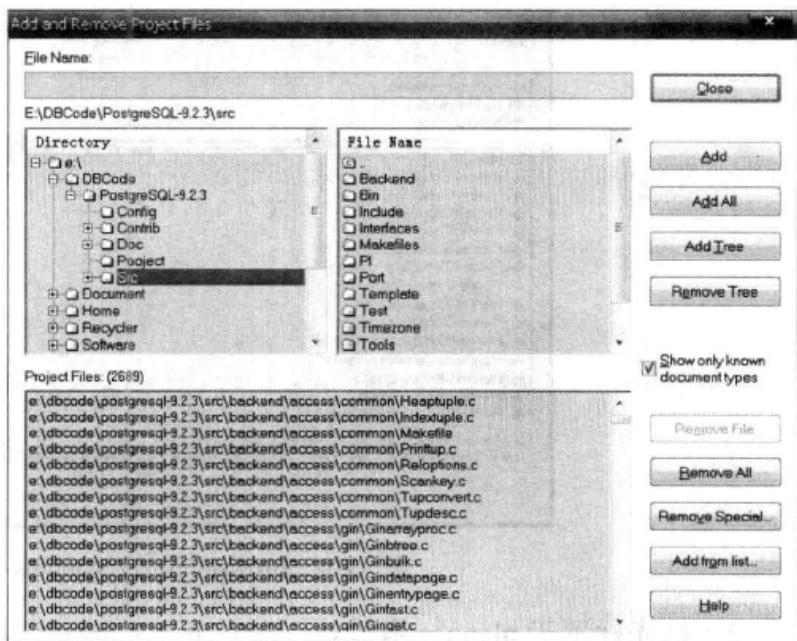


图 A-8 Source Insight 新建工程导入源码后的界面



图 A-9 Source Insight 建立文件的索引的界面

4. 阅读方式

1) 先从查询优化器的入口函数 planner 读起，可以按 F7 键，弹出 Browse Project Symbols 对话框。在 Symbol 文本框中输入 planner，下拉列表出现以 PI 开头的一些符号对象，其中有 planner 被选中，表示这个 planner 存在。单击 Jump 按钮，跳转到 planner 函数所在的代码，如图 A-10 所示。

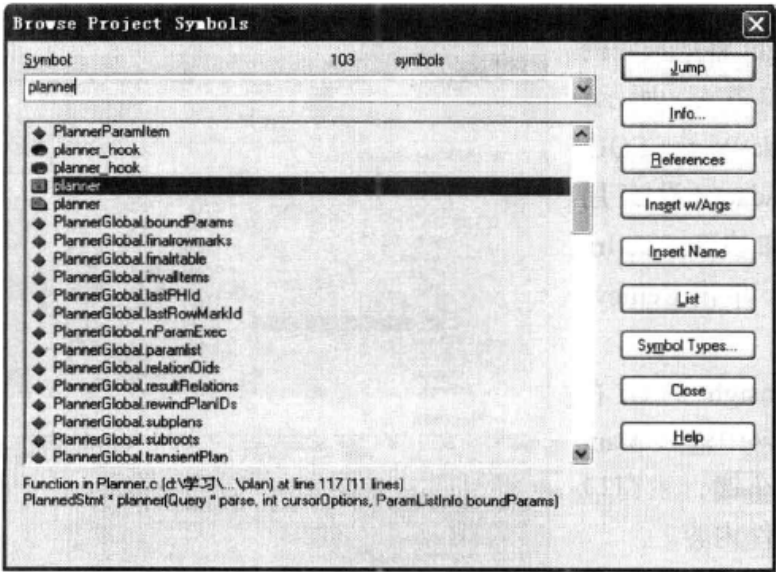


图 A-10 Browse Project Symbols 对话框

2) 把光标置于 planner 函数处，在左下方 Context Window 中显示了 planner 函数的定义，右下方 Relation Window 中显示了 planner 函数的调用关系，如图 A-11 所示。

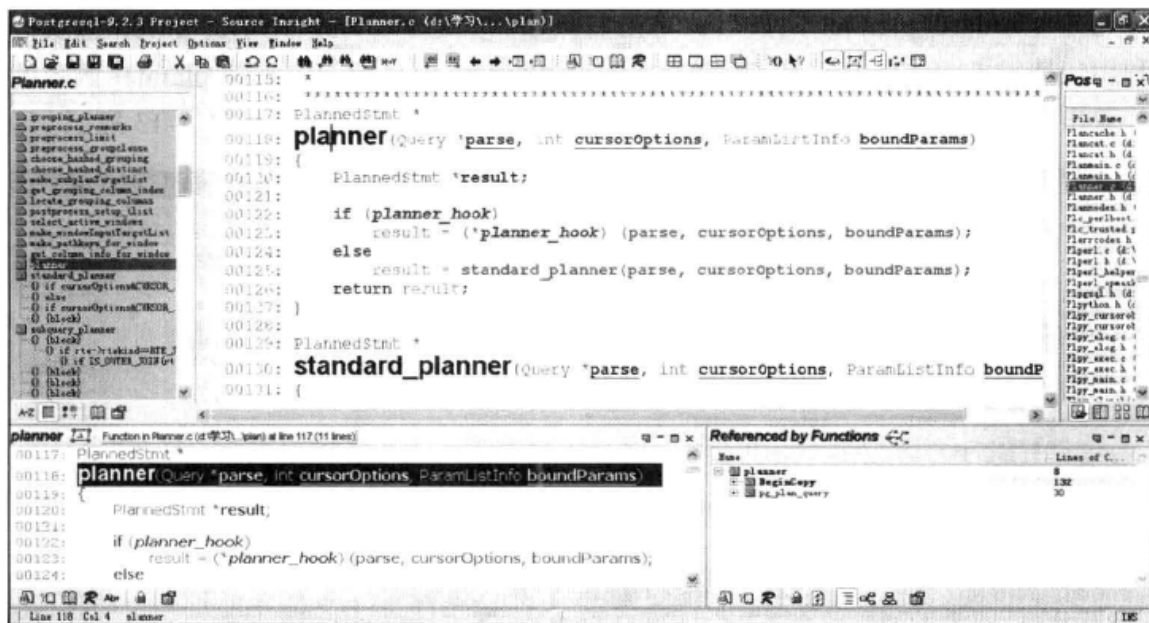


图 A-11 函数关系图

3) 如果光标在 `planner` 函数的 `PlannedStmt` 结构体位置, 左下方 Context Window 中显示了 `PlannedStmt` 结构体的定义, 右下方 Relation Window 中显示了 `PlannedStmt` 结构体的调用关系, 如图 A-12 所示。

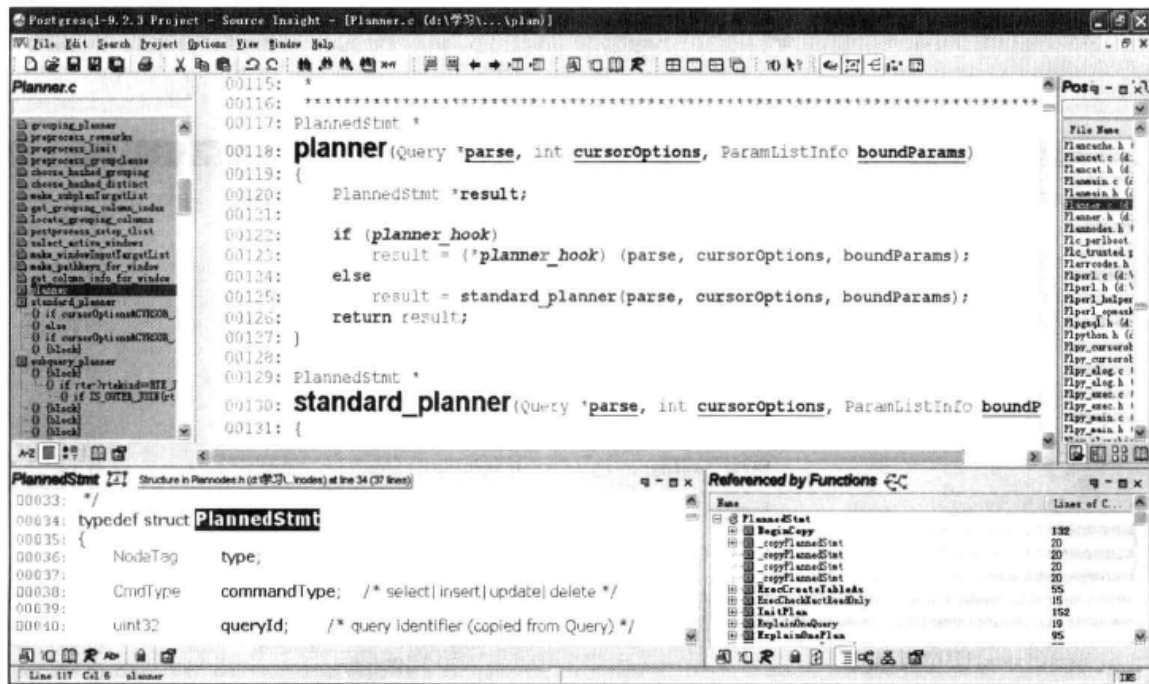


图 A-12 结构体关系图

4) 如果光标在 `planner` 函数的 `standard_planner` 函数位置, 左下方 Context Window 中显示了 `standard_planner` 函数的定义。这时, 查阅 `standard_planner` 函数的定义有 3 种方式:

一是在选中 `standard_planner` 函数后，单击右键菜单，选择 `Jump To Definition` 选项；二是直接利用快捷键 `Ctrl +=`；三是光标在 `Context Window` 中显示 `standard_planner` 函数的定义窗口，双击鼠标左键，则主窗口显示 `standard_planner` 函数的定义，如图 A-13 所示。

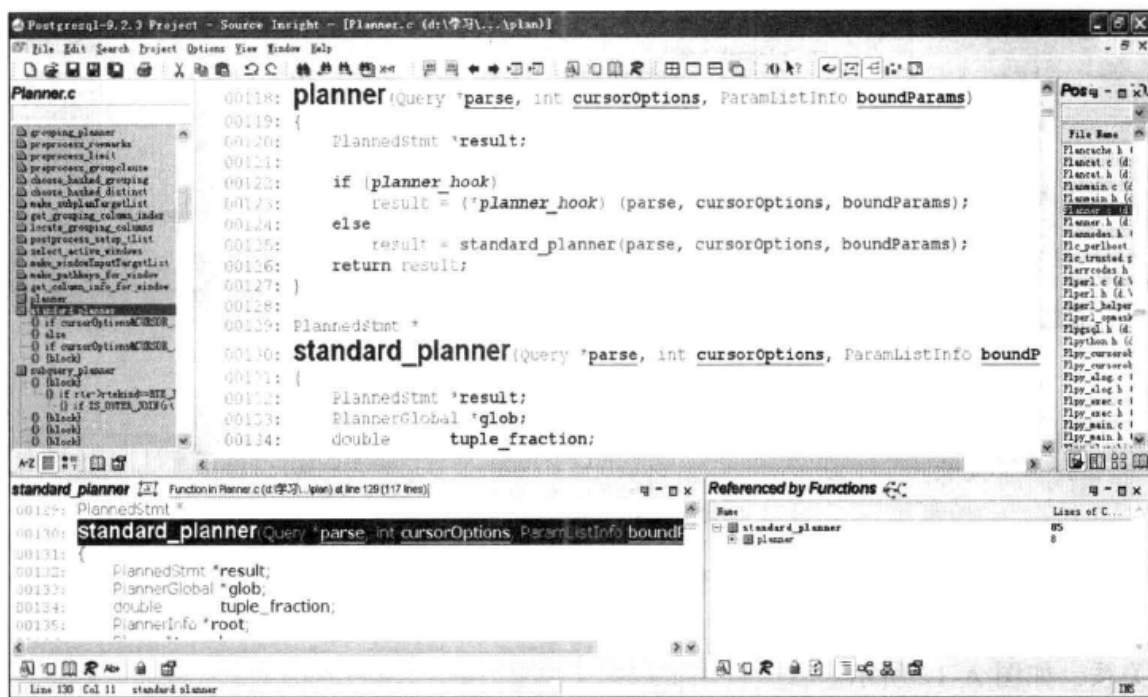


图 A-13 主窗口图显示函数定义

5) 查看结构体 `PlannedStmt` 定义的方式同上，如图 A-14 所示。

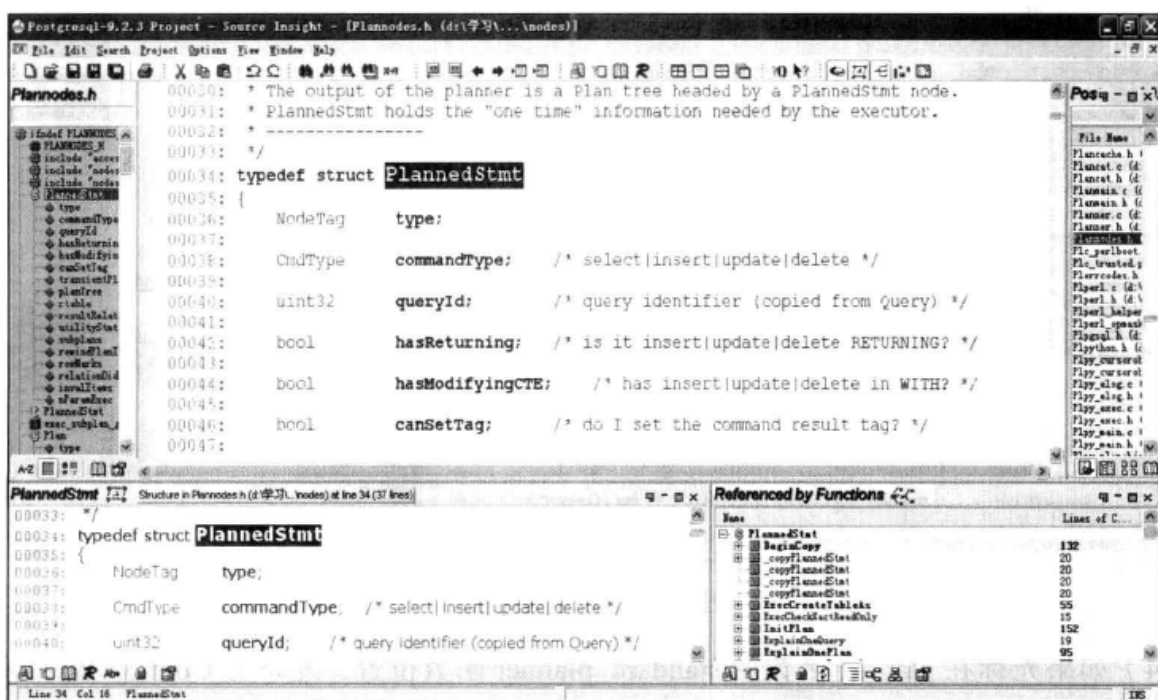


图 A-14 主窗口图显示结构体定义

6) 通常的方式,在主窗口阅读当前正在关注的代码,比如函数、结构体、变量或注释等;在左下方 Context Window 窗口中查看主窗口中选中的函数、变量等的定义。光标在主窗口和 Context Window 窗口间交互,阅读关注的代码。

7) 在图 A-15 中,左侧窗口显示的是当前主窗口中选中对象的框架,如选中的是函数,则显示函数的变量和主要代码结构,这个窗口有助于认识对象的整体情况。这要保证左侧窗口的属性信息选取了图 A-16 中的 Show local definitions 选项。

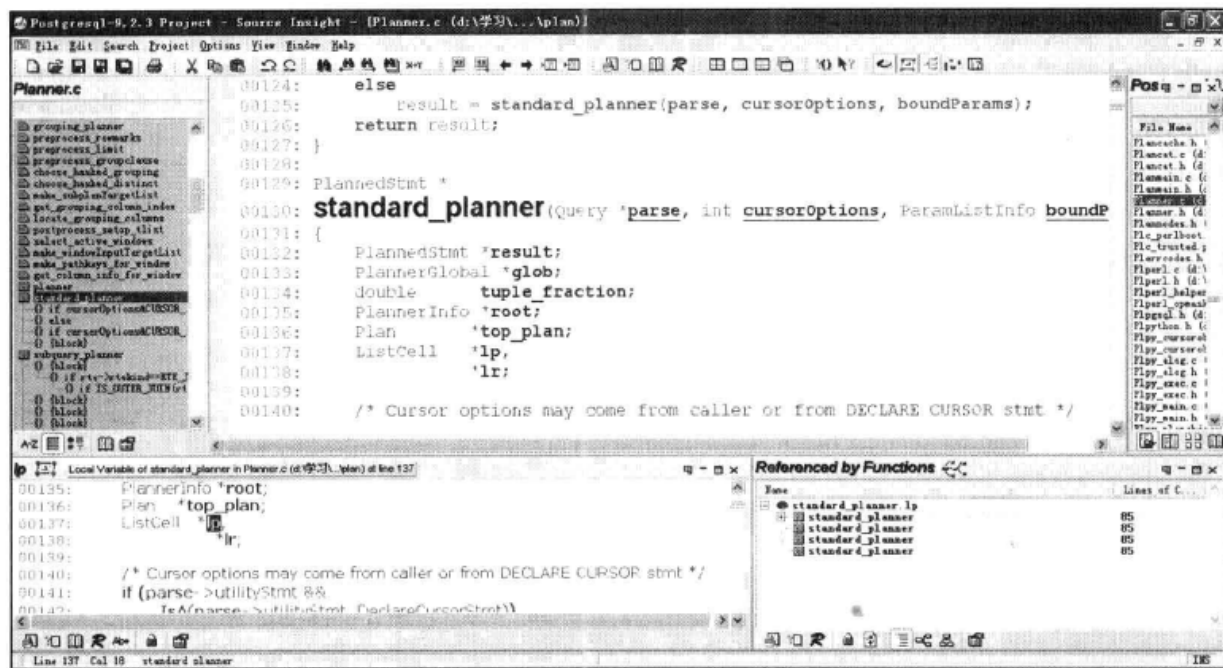


图 A-15 主窗口内容与左侧窗口关系图

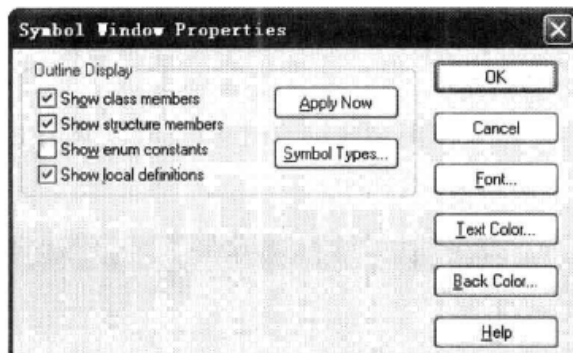


图 A-16 左侧窗口属性设置

5. 常用技巧

Q: 如何分析一个变量以理解其作用?

A: 如果掌握了一个变量的用法,则可以理解其作用。在 Source Insight 中,如阅读 `parse` 变量的作用,可利用以下方式:

- ❑ 快捷键 Shift + F8，高亮变量。可以突出显示代码中使用的这个变量，有助于理解。
- ❑ 选中变量 parse，在 View Relationship 窗口中显示变量的调用关系和调用位置；单击 View Relationship 窗口中显示的如“line 196”，在 Context Window 窗口中代码即更新为“line 196”指向的第 196 行的代码，这样，逐个单击 View Relationship 窗口显示的 parse 变量调用位置，可以查看变量的使用方式，如图 A-17 所示。

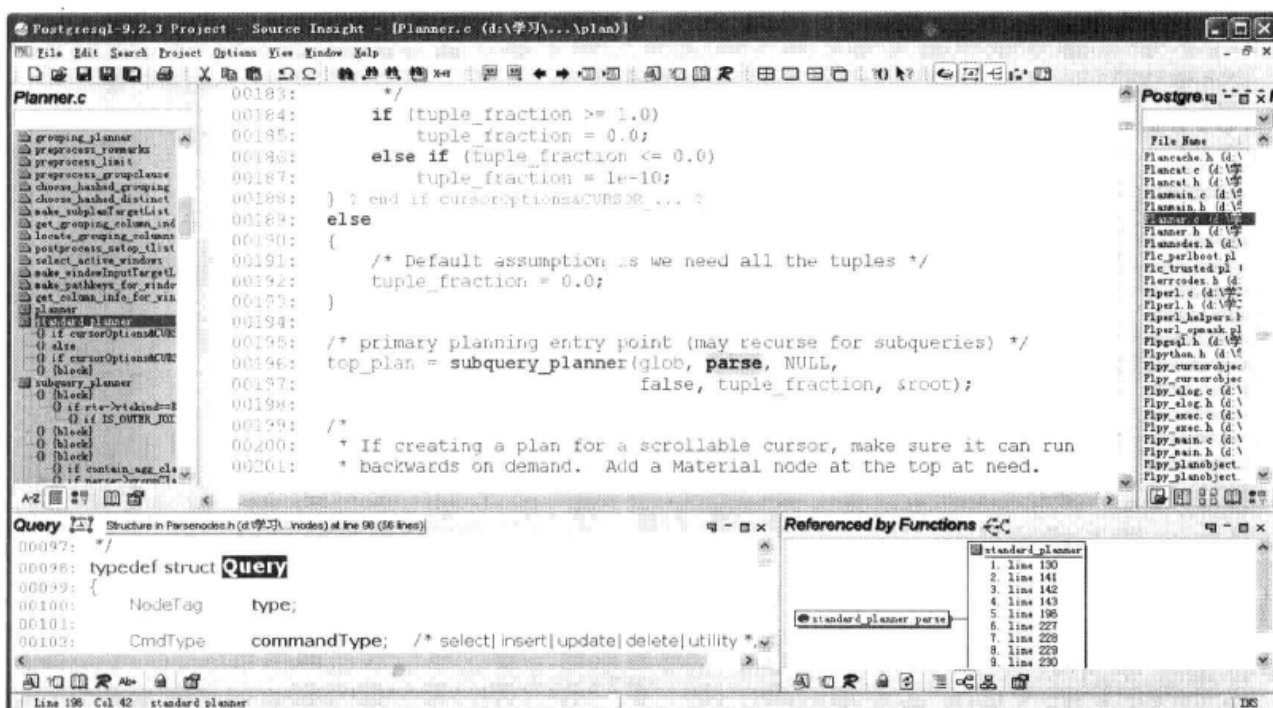


图 A-17 如何理解变量的作用图

- ❑ 本例变量 parse 是入口参数，还需要查看调用 standard_planner 函数的父函数，明确父函数如何传值给 standard_planner 函数的 parse 变量。

Q：如何快速查找一个关键字？

A：焦点（光标）在主窗口，使用快捷键 Ctrl + /，可以调用 Lookup References 对话框，完成对关键字的快速查找，如图 A-18 所示。

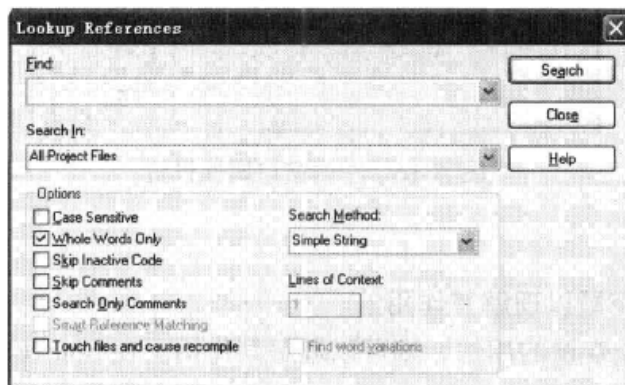


图 A-18 快速查找对话框

Q: 如何前翻、后翻看过的或准备再读的代码?

A: 使用快捷键, 前翻为“Alt + ,”; 后翻为“Alt + .”。

还可以使用工具条中的椭圆框中的按钮, 如图 A-19 所示。

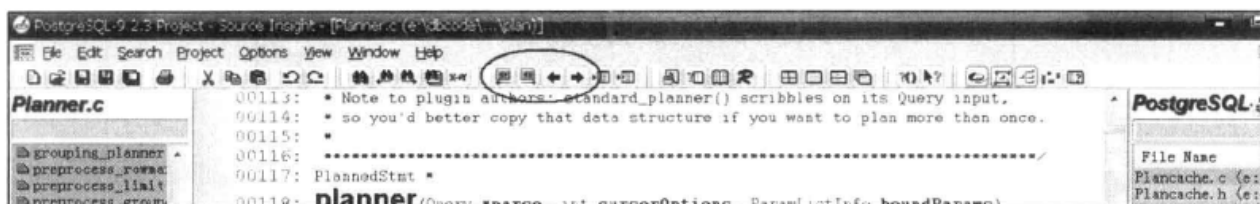


图 A-19 工具条中的前翻、后翻按钮

A.1.2 Eclipse

Eclipse 是优秀的跨平台的自由集成开发环境 (IDE)。Eclipse 本身是一个框架平台, 但由于众多插件的支持使得 Eclipse 能够支持其他多种语言, 如 C、C++ 等。

下面就以 MySQL 为例来介绍一下如何使用 Eclipse 阅读代码。

1. 准备工作

- 1) 下载 JDK, 安装 Java。
- 2) 下载 Eclipse C/C++ 版本, 如解压到 /usr/local/eclipse 下 (Eclipse IDE for C/C++ Developers)。下载地址如下:

<http://www.eclipse.org/downloads/>

2. 建立工程

- 1) 启动 Eclipse, 如图 A-20 所示。

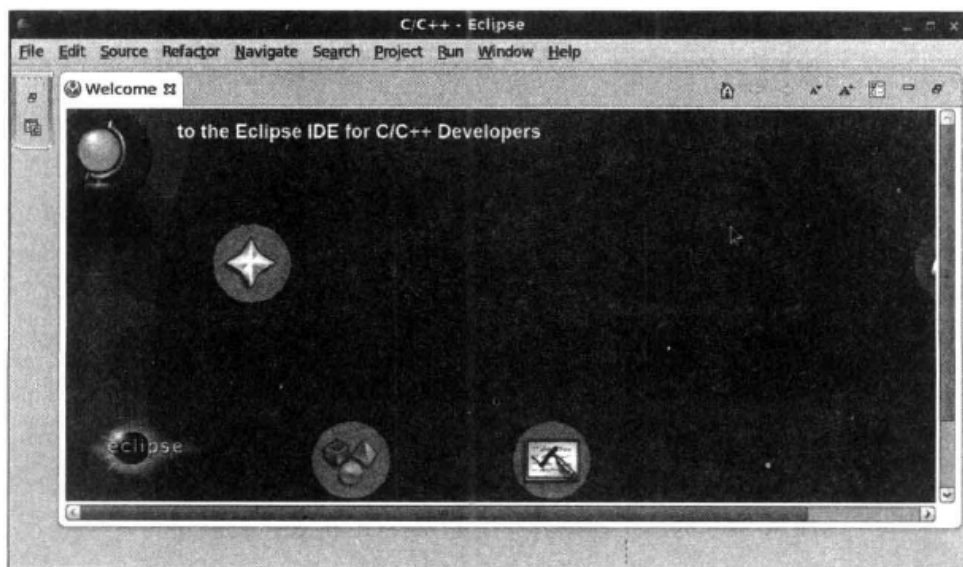


图 A-20 启动 Eclipse for C/C++ 版本

2) 在菜单中依次选择 File → New → Other 选项，建立新工程，选择 Makefile Project with Existing Code 选项，然后单击 Next 按钮，如图 A-21 所示。

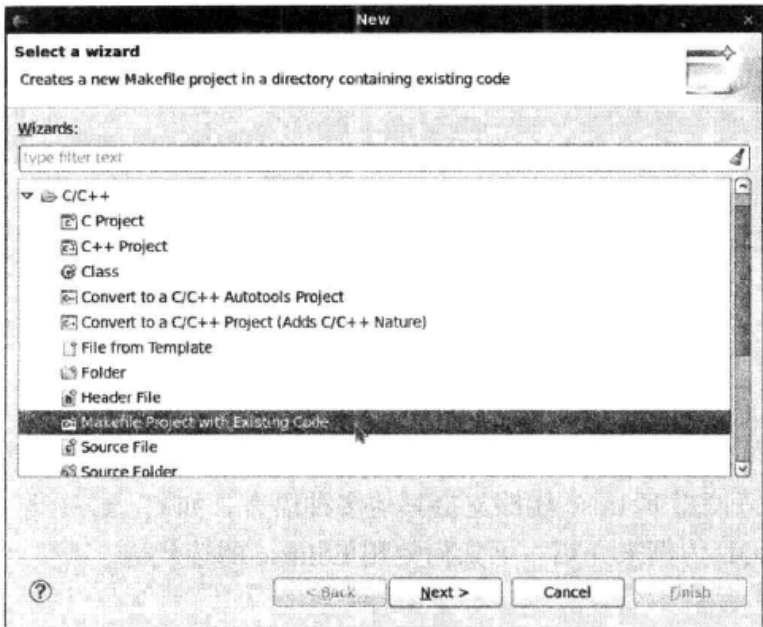


图 A-21 建立新工程

3) 在接下来的图形界面中输入工程名称等信息，然后单击 Finish 按钮，如图 A-22 所示。



图 A-22 为新工程命名并指定 MySQL 的源代码位置

4) 工程建立完毕，初始界面如图 A-23 所示。打开一个文件，如图 A-24 所示。

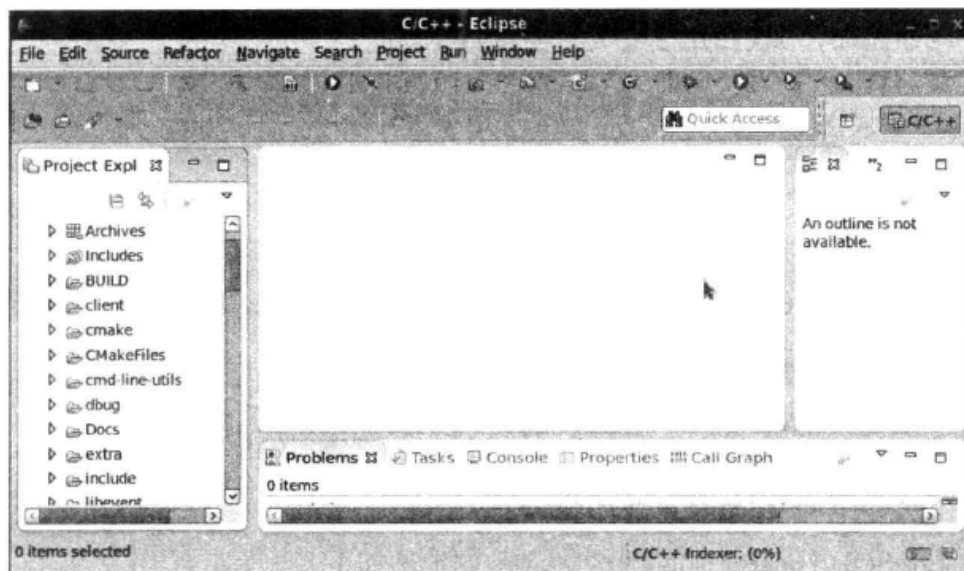


图 A-23 新工程建立完毕

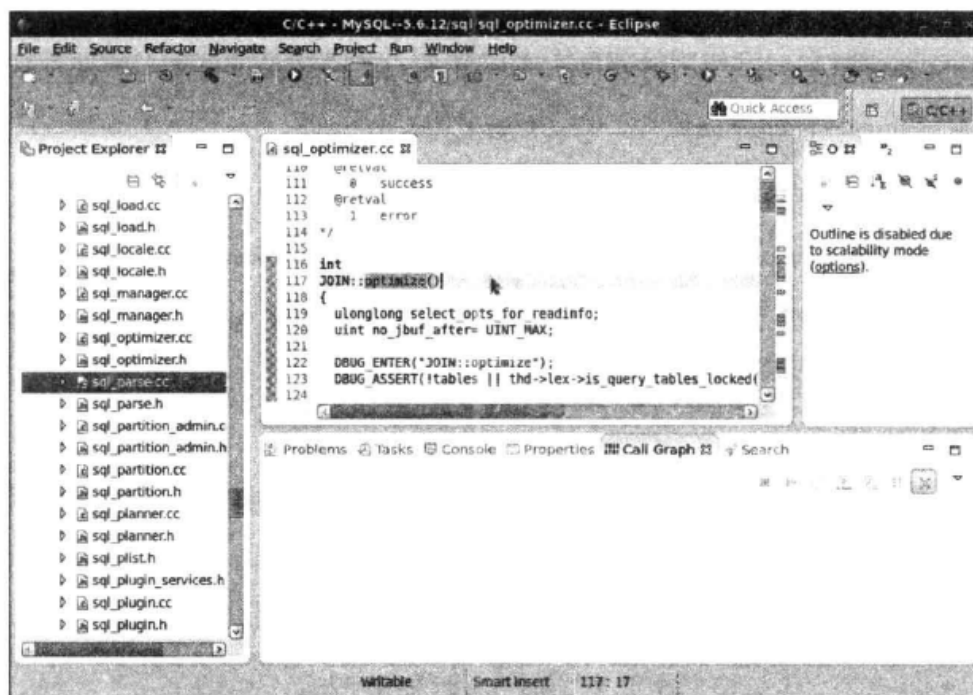


图 A-24 打开文件界面

3. 阅读方式

- 先从查询优化器的入口方法 JOIN.prepare 方法读起。可以打开文件 sql_resolver.cc，把光标定位到第 70 行。
- Eclipse 自身不带 UML 建模工具，建议下载 Eclipse 的 UML 相关插件，辅助代码阅读。

A.2 代码调试，最佳的动态理解方式

学习代码的最好方式，除了静态阅读外，还需要通过动态调试，查看处于运行态的数据库的执行过程、变量内存状态、函数栈调用关系等。

A.2.1 代码编译

下面介绍如何在 Windows 和 Linux 系统下编译 PostgreSQL 和 MySQL 的代码。

1. Windows 下编译 PostgreSQL

使用 Windows XP 版本编译 PostgreSQL V9.2.3 的步骤如下。

第一步：准备软件和环境

(1) 下载 PostgreSQL 等各类软件包

下载 PostgreSQL (postgresql-9.2.3.tar.gz) 的地址如下：

<http://www.postgresql.org/download/>

下载 Microsoft Windows SDK (如果新的 SDK 中包括了编译器和构建工具，则不需要下载 VC 的 express 版本。可使用最新的版本以避免一些不必要的问题) 的地址如下：

<http://www.microsoft.com/downloads/>

微软 VC 下载 (express 版够用且免费授权) 的地址如下：

<http://www.microsoft.com/visualstudio/eng/products/visual-studio-express-products>

下载 ActiveState Perl (推荐 5.8 以上版本，因 PostgreSQL V9.2.3 Windows 版本的编译脚本使用 Perl 5.7 之后的某个新增函数库) 的地址如下：

<http://www.activestate.com>

下载 Bison (版本为 Bison V1.875 或 V2.2 或之后的版本) 和 Flex (版本为 Flex V2.5.31 或之后的版本) 的地址如下：

<http://gnuwin32.sourceforge.net>

<http://www.postgresql.org/ftp/misc/winflex/>

如作者安装的 Bison、Flex (flex 版本低于 PostgreSQL 官方推荐版本，编译成功) 如下：

```
D:\DB\postgresql-9.2.3\src\tools\msvc>bison -V
bison (GNU Bison) 1.875
Written by Robert Corbett and Richard Stallman.
```

```
Copyright (C) 2002 Free Software Foundation, Inc.
```

```
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```
D:\DB\postgresql-9.2.3\src\tools\msvc>flex -V
flex version 2.5.4
```

下载 OpenSSL (PostgreSQL 依赖包) 的地址如下:

<http://slproweb.com/products/Win32OpenSSL.html>

下载 gettext (PostgreSQL 依赖包) 的地址如下:

<http://gnuwin32.sourceforge.net>

下载 MIT Kerberos (PostgreSQL 依赖包) 的地址如下:

<http://web.mit.edu/Kerberos/dist/index.html>

下载 libxml2 (依赖 iconv) 和 libxslt (PostgreSQL 依赖包) 的地址如下:

<http://zlatkovic.com/pub/libxml> (二进制)

<http://xmlsoft.org.openssl> (或源文件)

下载 ossp-uuid (PostgreSQL 依赖包) 的地址如下:

<http://www.ossp.org/pkg/lib/uuid/>

下载 Python (PostgreSQL 依赖包) 的地址如下:

<http://www.python.org>

下载 zlib (PostgreSQL 的逻辑备份还原工具编译使用, pg_dump/pg_restore) 的地址如下:

<http://www.zlib.net>

下载 Diff (目的是运行 PostgreSQL 的回归测试用例, 位于源码 src\test\regress 处) 的地址如下:

<http://gnuwin32.sourceforge.net>

(2) 配置软件

为 msbuild.exe 设置环境变量 PATH, 如:

C:\WINNT\Microsoft.NET\Framework\v2.0.50727;

为 vcbuild.exe 设置环境变量 PATH, 如:

D:\Microsoft Visual Studio 10.0\VC\vcpackages

假设 MinGW 等安装到 C:\Mysoft 下, 设置环境变量 PATH, 添加 PATH, 如:

C:\Mysoft\Mingw\bin;C:\Mysoft\MSYS\bin;C:\Mysoft\Perl\bin;C:\Mysoft\GnuWin32\bin;

解压 postgresql-9.2.3.tar.gz 到 D:\DB\postgresql-9.2.3 下, 在 src\tools\msvc\config_default.pl 文件中配置, 假设 PostgreSQL 依赖的包下载且安装到了 D:\DB\pg923-build 中, 则 config_default.pl 文件的内容可修改如下:

```
# Configuration arguments for vcbuild.
use strict;
```

```

use warnings;

our $config = {
    asserts=>0,# --enable-cassert
    # integer_datetimes=>1,# --enable-integer-datetimes - on is now default
    # float4byval=>1,# --disable-float4-byval, on by default
    # float8byval=>0,# --disable-float8-byval, off by default
    # blocksize => 8,# --with-blocksize, 8kB by default
    # wal_blocksize => 8, # --with-wal-blocksize, 8kb by default
    # wal_segsize => 16,# --with-wal-segsize, 16MB by default
    ldap=>1, # --with-ldap
    nls=>"D:\\DB\\pg923-build\\gettext",# --enable-nls=<path>
    tcl=>undef,# --with-tls=<path>
    perl=>"D:\\DB\\pg923-build\\perl",# --with-perl
    python=>"D:\\DB\\pg923-build\\python",# --with-python=<path>
    krb5=>undef,# --with-krb5=<path>
    openssl=>"D:\\DB\\pg923-build\\OpenSSL",# --with-ssl=<path>
    uuid=>"D:\\DB\\pg923-build\\uuid",# --with-oss-p-uuid
    xml=>"D:\\DB\\pg923-build\\libxml2",# --with-libxml=<path>
    xslt=>"D:\\DB\\pg923-build\\libxslt",# --with-libxslt=<path>
    iconv=>"D:\\DB\\pg923-build\\iconv",# (not in configure, path to iconv)
    zlib=>"D:\\DB\\pg923-build\\zlib"# --with-zlib=<path>
};

1;

```

(3) 修改 PostgreSQL 的代码

编辑器打开 src/backend/main/main.c, 找到 main 函数 (第 168 行), 在行首增加注释符号 “//”, 目的是在 Windows 上去掉对于超级用户的限制, 这样 PostgreSQL 就能在任何用户下运行了 (仅为开发调试方便, 不建议在生产环境这样使用)。

```

/*
 * Make sure we are not running as root.
 */
// check_root(progname);

```

第二步：编译、安装、初始化、运行 PostgreSQL

1) 启动命令行 (使用 Visual Studio 的命令提示行): Visual Studio 命令提示 (2010), 而不是 Windows XP 自带的命令行, 执行如下命令, 进行 DEBUG 版本的编译。

```

D:\Microsoft Visual Studio 10.0\VC>>d:
D:\>cd D:\DB\postgresql-9.2.3\src\tools\msvc
D:\DB\postgresql-9.2.3\src\tools\msvc>
D:\DB\postgresql-9.2.3\src\tools\msvc>build.bat DEBUG

```

2) 当编译完成后, 可以在命令行执行如下命令安装到 D:\exec\pg923:

```
perl install.pl D:\exec\pg923
```

3) 初始化数据目录的命令如下:

```
D:\exec\pg923\bin>initdb -UU -D ../data
```

4) 在初始化后的数据目录上以 33333 端口启动 PostgreSQL 实例, 命令如下:

```
D:\exec\pg923\bin>postgres -D ../data -p 33333
```

5) 用命令行交互式 SQL 工具连接 PostgreSQL 实例, 命令如下:

```
D:\exec\pg923\bin>psql -UU -p 33333 postgres
```

```
D:\exec\pg923\bin>psql -UU -p33333 template1
```

```
psql (9.2.3)
```

```
Type "help" for help.
```

```
template1=# SELECT datname FROM pg_database;
```

```
datname
```

```
-----
```

```
template1
```

```
template0
```

```
postgres
```

```
(3 rows)
```

第三步: 编译 PostgreSQL 可能遇到的问题

在命令行下执行编译 PostgreSQL 的命令如下:

```
D:\DB\postgresql-9.2.3\src\tools\msvc>build DEBUG
```

```
Unable to determine Visual Studio version: The nmake version could not be determined. at src/tools/msvc/Mkvcbuild.pm line 54.
```

如上出现错误, 表明 PostgreSQL 提供的脚本不能正常编译。

为了能正确编译, 需要对 PostgreSQL 的编译脚本进行修改, 修改方式如下:

修改 src\tools\msvc\Mkvcbuild.pm 文件第 54 行。

```
my $vsVersion = DetermineVisualStudioVersion();
```

修改后变为:

```
my $vsVersion;
```

修改 src\tools\msvc\VSObjectFactory.pm 文件第 26 行, 即增加如下命令 (本机使用 VS2010):

```
return new VS2010Solution(@_);
```

修改后变为:

```
my $visualStudioVersion = shift;
```

```
return new VS2010Solution(@_);
```

在 Windows 操作系统下编译 PostgreSQL 的其他信息, 可参考如下内容:

<http://www.postgresql.org/docs/current/static/install-windows.html>

2. Linux 下编译 MySQL

使用 Red Hat Enterprise 6.1 版本编译 MySQL V5.6.12 的步骤如下。

第一步：准备软件和环境

1) 检查操作系统的信息，命令如下：

```
[root@localhost /]# more /proc/version
more /proc/version
Linux version 2.6.32-131.0.15.el6.i686 (mockbuild@x86-001.build.bos.redhat.com)
(gcc version 4.4.4 20100726 (Red Hat 4.4.4-13) (GCC) ) #1 SMP Tue May 10 15:42:28 EDT 2011
```

2) 检查 MySQL 依赖的软件包（如果没有 make，请自行下载）。

验证 make 的版本信息，命令如下：

```
[root@localhost /]# make --version
make --version
GNU Make 3.81
Copyright (C) 2006 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.
```

This program built for i386-redhat-linux-gnu

验证 gcc 的版本信息，命令如下（表明 gcc 不存在）：

```
[root@localhost cmake-2.8.11.1]# gcc --version
gcc --version
bash: gcc: command not found
printf "\033[0;%s@%s:%s\007" "${USER}" "${HOSTNAME%%.*}" "${PWD/#$HOME/~}"
```

3) 下载 MySQL 等各类软件包。

下载 MySQL (mysql-5.6.12.tar.gz) 的地址如下：

<http://dev.mysql.com/downloads/mysql/#downloads>

下载 bison (bison-2.7.tar.gz) 的地址如下：

<http://www.gnu.org/software/bison/>

下载 cmake (cmake-2.8.11.1.tar.gz) 地址如下：

<http://www.cmake.org/cmake/resources/software.html>

下载 ncurses (ncurses-5.9.tar.gz) 的地址如下：

<http://ftp.gnu.org/pub/gnu/ncurses/>

下载 make 的地址如下：

<http://www.gnu.org/software/make/>

下载 gcc、gcc-c++ 的地址如下:

<http://www.gnu.org/software/gcc>

第二步: 编译前的其他准备——创建用户、目录、依赖软件安装等

1) 创建 MySQL 安装目录和数据存放目录, 命令如下:

```
mkdir -p /usr/local/db/mysql/
mkdir -p /db/data/mysql/
```

2) 使用 Linux 命令新建用户和用户组, 并赋予权限, 命令如下:

```
groupadd mysql
useradd -g mysql mysql
chown mysql.mysql -R /db/data/mysql/
```

3) 安装软件包 (注意如下安装顺序), 过程如下:

步骤 1 安装 gcc/gcc-c++ (最好是安装 gcc、gcc-c++ 的 rpm 包, 不建议编译 gcc)

- ① 解压 tar gcc-4.8.1.tar.gz;
- ② 进入 cd gcc-4.8.1 目录;
- ③ 执行 ./configure 命令;
- ④ 执行 make 命令;
- ⑤ 执行 make install 安装命令;
- ⑥ 同理安装 gcc-c++。

步骤 2 安装 bison

- ① 解压 tar bison-2.7.tar.gz;
- ② 进入 cd bison-2.7 目录;
- ③ 执行 ./configure 命令;
- ④ 执行 make 命令;
- ⑤ 执行 make install 安装命令。

步骤 3 安装 cmake

- ① 解压 tar zxvf cmake-2.8.11.1.tar.gz;
- ② 进入 cd cmake-2.8.11.1 目录;
- ③ 执行 ./configure 命令;
- ④ 执行 make 命令。

步骤 4 安装 ncurses

- ① 解压 tar ncurses-5.9.tar.gz;
- ② 进入 cd ncurses-5.9 目录;
- ③ 执行 ./configure 命令;
- ④ 执行 make 命令;
- ⑤ 执行 make install 安装命令。

第三步：编译、使用 MySQL 等

1) 编译 MySQL，具体如下。

解压，命令如下：

```
tar mysql-5.6.12.tar.gz
```

进入目录，命令如下：

```
cd mysql-5.6.12
```

为编译设置必要的参数，执行如下命令：

```
cmake .
\DCMAKE_INSTALL_PREFIX=/usr/local/db/mysql/
\DMYSQL_DATADIR=/db/data/mysql
\DMYSQL_UNIX_ADDR=/db/data/mysql/mysql.sock
\DWITH_INNOBASE_STORAGE_ENGINE=1
\DENABLED_LOCAL_INFILE=1 \DMYSQL_TCP_PORT=3306
\DEXTRA_CHARSETS=all \DDEFAULT_CHARSET=utf8
\DDEFAULT_COLLATION=utf8-general_ci
\DMYSQL_UNIX_ADDR=/db/data/mysql/mysql.sock
\DMYSQL_USER=mysql \DWITH_DEBUG=0
```

编译源码，执行命令：

```
make
```

执行安装，命令如下：

```
make install
```

2) 配置 MySQL，命令如下：

```
[root@localhost mysql-5.6.12]# cp support-files/my-default.cnf /etc/my.cnf
[root@localhost mysql-5.6.12]# cp support-files/mysql.server /etc/init.d/mysqld
[root@localhost mysql-5.6.12]# chmod 755 /etc/init.d/mysqld
```

3) 初始化数据库，命令如下：

```
[root@localhost mysql]scripts/mysql_install_db --user=mysql --basedir=/usr/local/db/mysql --datadir=/db/data/mysql/
```

4) 启动 MySQL 服务，并检查 MySQL 服务是否启动；如果是启动 MySQL 的服务，则用 MySQL 命令行连接服务器，初始密码为空。

```
[root@localhost mysql]#/etc/init.d/mysqld start
```

检查 MySQL 服务是否启动（启动成功，则可以通过如下命令发现 mysqld 进程存在）。

```
[root@localhost mysql-5.6.12]#ps -ef |grep mysql
```

5) 启动客户端 MySQL 命令行工具，修改 root 密码，命令如下：

```
[root@localhost mysql-5.6.12]#/usr/local/db/mysql/bin/mysql -u root -p
```

```
mysql> use mysql;
mysql> update user set password=PASSWORD("mysqldba") where user='root';
mysql> FLUSH PRIVILEGES;
```

A.2.2 代码跟踪

在想要跟踪之处打上断点，可以进行代码跟踪。各个调试根据的跟踪原理和方式相似，不再赘述。请在互联网上查看相关资料。

1. Windows 下使用 Visual Studio 跟踪 PostgreSQL

当 build 工作完成后，在 D:\DB\postgresql-9.2.3 目录下会生成诸多的 Visual Studio 工程文件，如 postgres.vcxproj、psql.vcxproj 等，还有一个 pgsql.sln 文件，这是 PostgreSQL 的 build.bat 脚本在编译 PostgreSQL 的源代码后自动生成的工程文件。用 Visual Studio 启动 pgsql.sln 文件，在图形化的界面中调试 PostgreSQL 是一种方式。

本节介绍另外一种方式，使用 VS 加载运行中的 postgres 进程进行调试。

在使用 build DEBUG 命令编译 DEBUG 版本后，执行了安装命令和初始化命令（参见“Windows 下编译 PostgreSQL”一节），二进制文件和数据命令结构如图 A-25 所示。

因为 PostgreSQL 的 DEBUG 版本总带有符号表（symbols 目录），这成为调试 PostgreSQL 的重要依据（进入调试模式后，调试器将自动加载符号文件）。

在命令行执行如下命令启动 PostgreSQL：

```
D:\exec\pg923\bin>postgres -p33333 -D ../data
LOG: database system was interrupted; last known up at 2013-06-11 17:35:06 HKT
LOG: database system was not properly shut down; automatic recovery in progress
LOG: record with zero length at 0/174BDF8
LOG: redo is not required
LOG: autovacuum launcher started
LOG: database system is ready to accept connections
```

在命令行执行 psql，连接 PostgreSQL 的数据库服务器。

```
D:\exec\pg923\bin>psql -UU -p33333 postgres
psql (9.2.3)
Type "help" for help.

postgres=
```

在 data 目录下打开文件 postmaster.pid（PostgreSQL 运行时，会在数据目录下创建此文件，文件的首行是 PostgreSQL 主进程的 PID 号，用以标识此数据目录已经被 4567 进程使用，起到排斥新 PostgreSQL 进程使用此数据目录的作用——文件锁），内容如下：

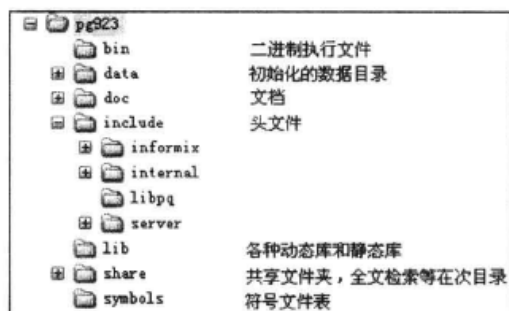


图 A-25 PostgreSQL 的 DEBUG 版本安装和初始化后的目录结构图

```
4567
D:/exec/pg923/bin/./data
1373507343
33333

localhost
```

在 Windows 任务管理器中，观察现有的 postgres 进程，然后启动一个客户端 psql，连接到 PostgreSQL 的 33333 端口上。

在 Windows 任务管理器中，观察所有的 postgres 进程，找到新增加的一个 postgres（如找到 PID 为 4567 的 postgres 进程是因客户端连接后新增的进程，其他许多 postgres 进程是本机安装了其他版本的 PostgreSQL）进程，如图 A-26 所示。在这个新增进程上右击，弹出右键快捷菜单，选择“调试(D)”选项，启动 Visual Studio（以下简称 VS）。



图 A-26 从 Windows 任务管理器调试 PostgreSQL 图

在 VS 启动后，在“解决方案”下的 postgres 上右击，选择右键快捷菜单中的“进入并单步执行新实例(I)”选项对 postgres 进行调试，如图 A-27 所示。

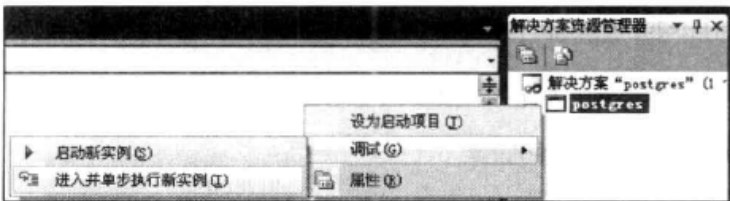


图 A-27 使用 VS 调试 postgres 进程

在 VS 中依次选择“文件(F)”→“打开(O)”→“文件(F)”，打开 planner.c 文件，在 subquery_planner 函数中打上断点，如图 A-28 所示。

在 psql 客户端行执行如下查询，VS 的断点被激活，逐步执行，查看执行过程，最后得

到查询结果如下：

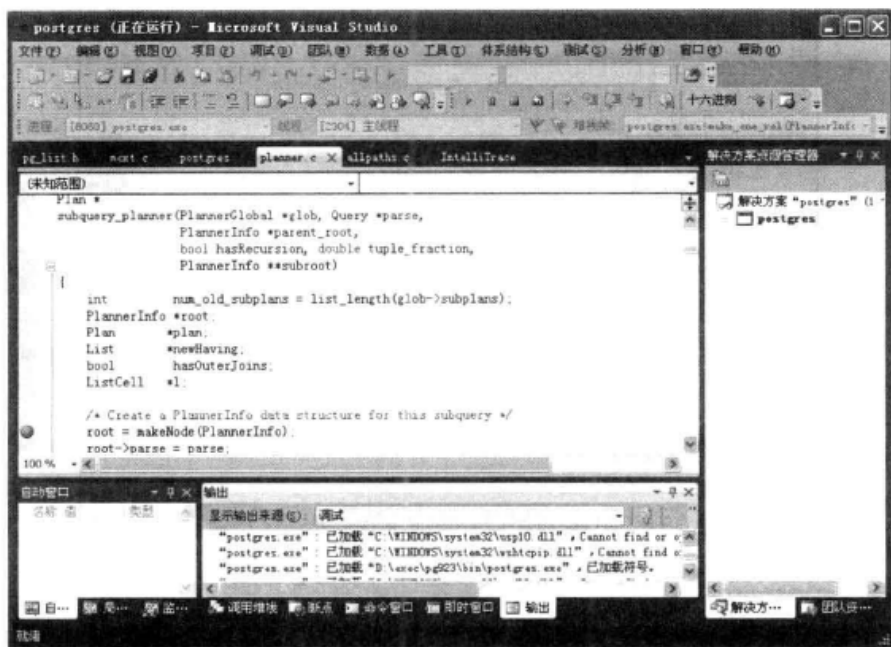


图 A-28 使用 VS 设置 postgres 进程断点

```
postgres=# SELECT relname,nspname FROM pg_class c, pg_namespace n WHERE c.relnamespace=n.oid LIMIT 2;
 relname | nspname
-----+-----
 pg_statistic | pg_catalog
 pg_type | pg_catalog
(2 rows)
```

2. Linux 下使用 Eclipse 跟踪 MySQL

Linux 下调试 MySQL，可以直接使用 gdb，也可以使用图形化的 Eclipse。本节介绍使用 Eclipse 跟踪 MySQL 的代码。

打开 Eclipse，依次选择菜单 Run → Debug Configurations，如有需要，可以在此配置一些信息（例如在图 A-30 可以单击 Variables 按钮参照 MySQL 的启动参数做配置），如图 A-29、图 A-30 所示。

参数配置好以后，打开文件 sql_resolver.cc，在第 86 行处打上断点，如图 A-31 所示。

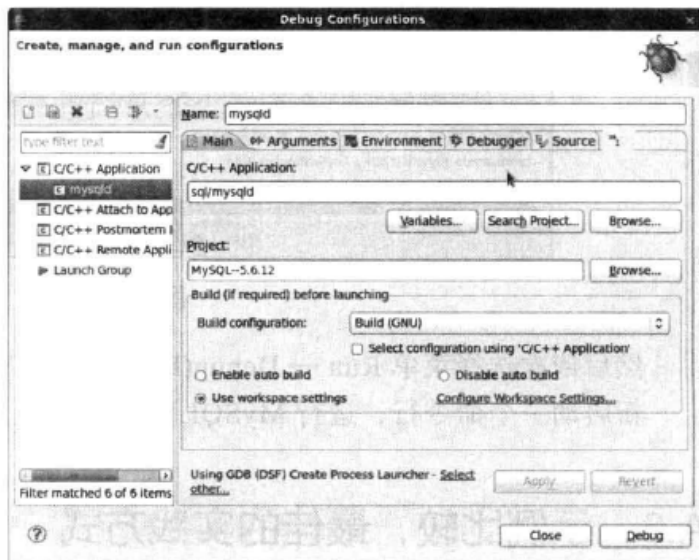


图 A-29 MySQL 入口函数运行配置

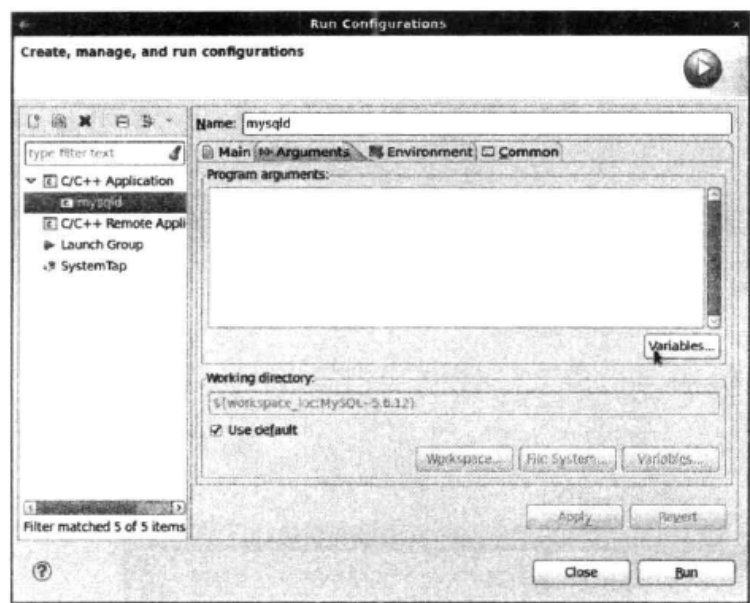


图 A-30 MySQL 运行启动参数配置

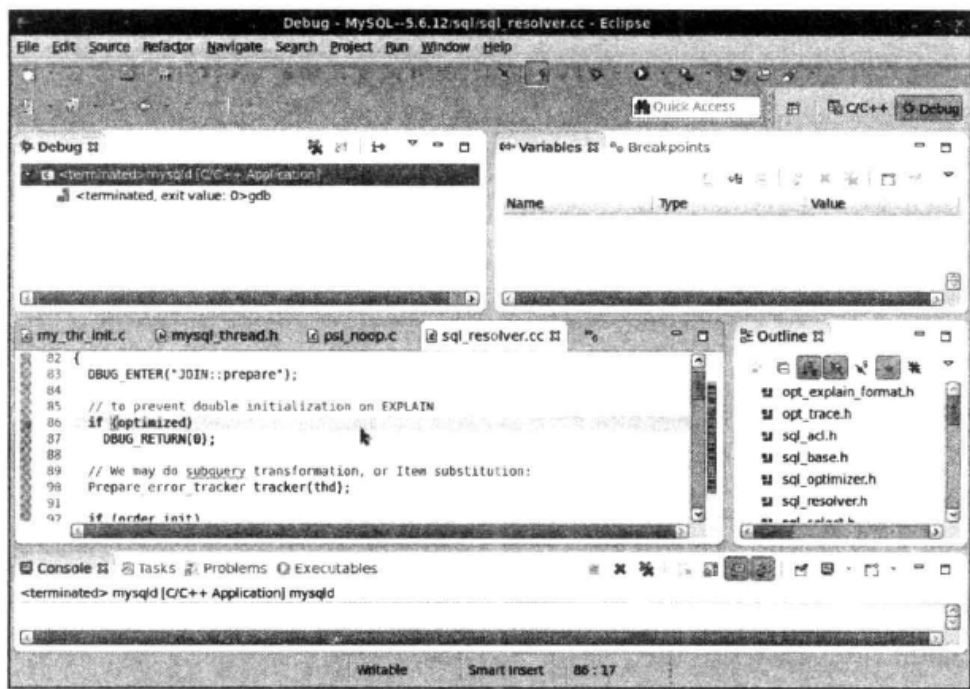


图 A-31 设置断点

然后依次选择菜单 Run → Debug(F11)，启动 MySQL 的调试过程。
新启动一个命令行，运行 MySQL 的客户端工具，执行查询语句即可。

A.3 示例比较，最佳的实践方式

通过 SQL 语句查看查询 SQL 的执行计划，可以更好地理解数据库查询优化器的工作过

程。同样的 SQL 语句，可以在 PostgreSQL 和 MySQL 中分别执行，通过对比，有助于加深理解和掌握。如果将示例执行和代码阅读、代码调试相结合，则效果更加。

A.4 思考总结，最佳的掌握方式

开卷有益，读书重要。通过代码阅读、示例比较、阅读各类书籍，系统掌握知识。但这些都如过眼烟云，很快就会淡忘，如果能不断总结，化繁为简，思考原理，则知识会变为自身的能力，永生跟随。

附录 B

如何阅读本书

本书是一本数据库内核相关书籍，从数据库的查询优化器入手，对数据库的查询优化引擎进行了分析和对比，对查询优化的技术做了全面的总结和剖析。从不同角度看，可能有着不同的感受；不同角色的人，可能对本书有着不同的需求。阅读本书，如果能和实践结合，则效果甚佳。

B.1 数据库引擎开发人员

如果您是一名数据库内核的开发人员，也许您对于 PostgreSQL 和 MySQL 查询优化器的代码结构、实现过程更为感兴趣。

本书的第二篇、第三篇，主要从 PostgreSQL 和 MySQL 的代码入手，依据实现流程进行分析，既有主干过程的描述，也有重要细节的剖析，更有重要话题的全流程总结、对比。

从目录结构出发，您可以一览 PostgreSQL 和 MySQL 查询优化器的全貌，抓住主干。

从每一个函数的解析中，您可以掌握 PostgreSQL 和 MySQL 查询优化器的实现细节，对查询优化器了然于胸。

从第二篇、第三篇和代码对比，您可以对比两个查询优化器实现的异同，品味它们相似而又不同的味道。从一座山中看另外一座山，从两山之外再看山，其中的滋味，非“只缘身在此山中”所能比拟。

B.2 数据库管理员

如果您是一名数据库管理员，也许您对于 PostgreSQL 和 MySQL 查询优化器所能支持的优化功能更感兴趣。

本书的第一篇给出了查询优化技术的全貌，有助于您对查询优化技术建立全面的概念，并能深入理解；第四篇主要从 PostgreSQL 和 MySQL 支持的优化功能入手，进行了总结对

比，有助于您对这两个数据库的查询优化功能全面而准确的掌握，知其所能，也知其所不能。这样在工作实践中，可更好地应用。

全书既有理论、又有代码分析，还有对比总结、示例实践，从多个角度帮助数据库管理员掌握数据库的 SQL 查询优化原理，从而能在实践中有效地指导 SQL 的优化工作。

阅读完本书，您最大的收获可能是“SQL 查询怎么写，效率才更高”，此后，SQL 优化工作，您不再需要发愁，本书会为您加油，为您助力。

B.3 数据库爱好者和学生

数据库技术的爱好者和学生是数据库技术发展的重要源泉。因为热爱，您能钻研；因为学习，您能深入。

全书共 4 篇，第一篇是理论、第二篇是 PostgreSQL 的查询优化器实现、第三篇是 MySQL 的查询优化器实现、第四篇是理论和 PostgreSQL 和 MySQL 查询优化技术的对比，全面学习这 4 部分内容，全面掌握这 4 部分内容，相信您能对数据库查询优化器和查询优化技术有全面而深刻的掌握。

从第一篇入手，学习查询优化的理论和技术，先对查询优化建立全面的印象和概念。

阅读第二篇内容，结合 PostgreSQL 的代码，互为印证。把本书所附示例逐个执行，结合原理、代码、示例执行结果，领悟 PostgreSQL 查询优化器的实现、SQL 优化的结果。

阅读第三篇内容，结合 MySQL 的代码，互为印证……

结合第二篇、第三篇内容，反复把原理、代码、示例融合……若能内外皆修必将形神具备。

请不要放弃，遇到困难，坚持必有所成。

附录 C

如何阅读查询执行计划

学习数据库查询优化技术，第一步需要看明白查询执行计划，根据查询执行计划理解查询优化器的执行过程，体会优化技术的运用情况。所以，读懂查询执行计划是掌握查询优化技术的必要条件。

以下从 PostgreSQL 和 MySQL 的查询执行计划的格式和关键字，介绍 PostgreSQL 和 MySQL 的查询执行计划，并结合实例，帮助读者理解查询执行计划。

示例 演示如何阅读 PostgreSQL 和 MySQL 的查询执行计划。先创建 5 张表，命令如下（各个表的数据量为：t1 表 10000 行数据，t2 表 100 行数据，t3 表 100 行数据，t4 表 7 行数据，t5 表 10 行数据）：

```
CREATE TABLE t1 (id1 INT, a1 INT, b1 INT, PRIMARY KEY(id1));
CREATE TABLE t2 (id2 INT, a2 INT, b2 INT);
CREATE TABLE t3 (id3 INT UNIQUE, a3 INT, b3 INT);
CREATE TABLE t4 (id4 INT, a4 INT, b4 INT);
CREATE TABLE t5 (id5 INT UNIQUE, a5 INT, b5 INT);
```

C.1 PostgreSQL 的查询执行计划解析

PostgreSQL V9.2.3 版本显示查询执行计划的函数入口是 ExplainQuery 函数。通过 EXPLAIN 这个 SQL 语句触发，获得执行。显示执行计划在新版本中的功能越来越丰富，不仅包括路径信息（单表扫描方式、两表连接算法、多表连接的顺序，每个步骤的花费、总的费用），而且包括了缓冲区的使用情况，并且提供多种输出格式，便于阅读，方便分析定位问题。

C.1.1 EXPLAIN 功能

语法格式：

```
EXPLAIN [( option [, ...] )] statement
```

EXPLAIN [ANALYZE] [VERBOSE] statement

可选项 option 包括：

ANALYZE [boolean]

VERBOSE [boolean]

COSTS [boolean]

BUFFERS [boolean]

TIMING [boolean]

FORMAT { TEXT | XML | JSON | YAML }

说明：

- ❑ EXPLAIN。显示查询执行计划，用以得到 statement 语句的执行计划。
- ❑ 参数 (option [, ...])。可选。其子项可以有多种。
- ❑ 参数 ANALYZE。可选。执行 ANALYZE 命令^①后，执行 EXPLAIN 命令，并显示 statement 的真实运行时间。ANALYZE 的布尔参数默认值是 FALSE。
- ❑ 参数 VERBOSE。可选。得到 statement 语句的执行计划和执行计划中的每个结点的详细信息。VERBOSE 的布尔参数默认值是 FALSE。
- ❑ 参数 COSTS。可选。得到计划中的每个结点中 cost、rows、widths 的估算值。COSTS 的布尔参数默认值是 TRUE。
- ❑ 参数 BUFFERS。ANALYZE 出现时可选。缓存的使用情况，包括：
 - 共享缓存块^② (shared blocks) 的 hit^③、read、dirtied^④、written^⑤ 数值。
 - 本地缓存块^⑥ (local blocks) 的 hit、read、dirtied、written 数值。
 - 临时块^⑦ (temp blocks) 的 read、written 数值。
- ❑ 参数 TIMING。ANALYZE 出现时可选。显示每个结点的启动时间和总时间花费。TIMING 的布尔参数默认值是 TRUE。
- ❑ 参数 FORMAT。可选。指定输出格式。可选的格式如下。
 - TEXT：默认值。普通文本格式，以行为单位，显示每个结点的计划信息，以缩进格式表示子结点的计划信息。BUFFERS 参数时的文本格式，只输出非零值。
 - XML：以 XML 格式显示执行计划。
 - JSON：以 JSON 格式显示执行计划。
 - YAML：以 YAML 格式显示执行计划。

① PG 支持独立的 ANALYZE 命令，用以统计收集数据库对象的各种数据。

② Shared blocks contain data from regular tables and indexes.

③ A hit means that a read was avoided because the block was found already in cache when needed.

④ dirtied indicates the number of previously unmodified blocks that were changed by this query.

⑤ written indicates the number of previously-dirtied blocks evicted from cache by this backend during query processing.

⑥ local blocks contain data from temporary tables and indexes.

⑦ temp blocks contain short-term working data used in sorts, hashes, Materialize plan nodes, and similar cases.

❑ **boolean**。值的写法，可以是：

- TRUE/ON/1
- FALSE/OFF/0

❑ **statement**。可被 EXPLAIN 执行的 SQL 语句，包括的类型有 SELECT、INSERT、UPDATE、DELETE、VALUES、EXECUTE、DECLARE、CREATE TABLE。

C.1.2 查询执行计划

理解 PostgreSQL 查询执行计划，需要理解执行顺序和结点解析两个部分。

1. 执行顺序

执行 5 表连接的查询语句，演示 PostgreSQL 的查询执行计划样式，语句如下：

```
EXPLAIN SELECT * FROM (t1 LEFT JOIN t2 ON true), (t3 FULL JOIN t4 ON true), t5 WHERE id1=id2 AND id2=id3 AND id3=id4 AND id4=id5;
```

执行 SQL，查询执行计划如图 C-1 所示。

1	QUERY PLAN
2	-----
3	Nested Loop (cost=6.43..43.40 rows=1 width=60)
4	-> Hash Join (cost=6.43..43.12 rows=1 width=48)
5	Hash Cond: (t4.id4 = t1.id1)
6	-> Seq Scan on t4 (cost=0.00..29.40 rows=1940 width=12)
7	-> Hash (cost=6.42..6.42 rows=1 width=36)
8	-> Merge Join (cost=5.39..6.42 rows=1 width=36)
9	Merge Cond: (t1.id1 = t2.id2)
10	-> Merge Join (cost=0.07..25.60 rows=100 width=24)
11	Merge Cond: (t1.id1 = t3.id3)
12	-> Index Scan using t1_pkey on t1 (cost=0.00..328.25 rows=10000 width=12)
13	-> Index Scan using t3_id3_key on t3 (cost=0.00..13.75 rows=100 width=12)
14	-> Sort (cost=5.32..5.57 rows=100 width=12)
15	Sort Key: t2.id2
16	-> Seq Scan on t2 (cost=0.00..2.00 rows=100 width=12)
17	-> Index Scan using t5_id5_key on t5 (cost=0.00..0.27 rows=1 width=12)
18	Index Cond: (id5 = t1.id1)
19	(16 行记录)

图 C-1 PostgreSQL 执行 SQL 的查询执行计划

下面对图 C-1 简要分析如下：

- ❑ 从第 1 行到第 19 行表示了完整的查询执行计划。
- ❑ 每一行为一个结点，行首表示本结点对应的操作，“on”表示作用的对象，行中的括号表示本结点的操作的花费（cost）、元组数（rows）、元组宽度（width）。
- ❑ 花费（cost）的格式：启动花费 + “..” + 总花费。
- ❑ 从第 3 行到第 19 行，逐行嵌套，表示了查询的执行顺序，嵌套层次最深的，最先执行。
- ❑ 同样嵌套深度的，从上到下，先予执行。
- ❑ 第 12、第 13 行最先执行，通过“归并连接（Merge Join）”连接算法得到中间件结果第 10 行；第 16 行执行后经排序操作得到第 14 行；然后两者（第 10 行、14 行）再做“归并连接（Merge Join）”得到第 8 行的结果。

□ 表的连接次序为：t1、t3、t2、t4、t5。这和初始给定的连接次序不同。

2. 结点解析

PostgreSQL 查询执行计划的各中结点的解释如表 C-1 所示。

表 C-1 PostgreSQL 结点解释表

类 型	结 点	说 明
扫描操作	Seq Scan	顺序扫描表
	Index Scan	索引扫描（读取索引文件，然后读取数据文件）
	Index Only Scan	索引只读扫描（只读取索引文件，不读取数据文件）
	Tid Scan	根据元组头中的 ctid 直接定位元组的位置，读取数据文件
	Bitmap Index Scan	位图索引扫描（利用索引，获得满足选择条件的元组的位图，直接读取索引文件，不读取数据文件）
	Bitmap Heap Scan	位图 Heap 索引扫描（利用 Bitmap Index Scan 得到的位图，获取元组，直接在数据文件中读取数据）
	Function Scan	函数扫描，处理范围表中有函数的情况（PostgreSQL 允许函数可以返回元组集合，类似表的元组集合）
	Subquery Scan	子查询扫描
	Values Scan	Values 扫描（PostgreSQL 提供了 Values 语句格式，允许 Values 链表可以返回元组集合，类似表的元组集合）
	CTE Scan	CTE 扫描（PostgreSQL 支持在 SELECT 查询中用 WITH 子句定义子查询，对 WITH 子句的扫描称为 CTE 扫描）
	WorkTable Scan	工作表扫描（与 RecursiveUnion 结点共同完成递归合并子查询）
连接操作	Nested [(type)] Loop	嵌套循环连接。type 值可能为 Inner、Left、Right、Full、Semi、Anti。Inner 的显示可省略
	Merge [(type)] Join	归并连接。type 值同上
	Hash [(type)] Join	哈希连接。type 值同上
物化操作	Material	物化操作（缓存子结点的结果，适用于需要多次扫描的结点）
	Unique	DISTINCT 操作（DISTINCT，对下层已经排序结点的输出去重）
	Group	分组操作（GROUPBY，对下层结点的输出分组）
	Sort	排序操作（ORDERBY，对下层结点的输出排序）
	Limit	LIMIT/OFFSET 操作（从下层结点的输出中选出一定范围的元组）
	Aggregate	带有聚集函数的 GROUPBY 操作
	GroupAggregate	分组聚集（带有分组操作的聚集，需要排序）
	HashAggregate	Hash 聚集（使用 Hash 算法完成聚集，不需要排序）
	WindowAgg	窗口函数
集合操作	Append	UNION/ INTERCECT/EXCEPT 操作（多个子查询的结合操作）
	SetOp [type]	INTERCECT/EXCEPT（需要排序）。type 的值可能为：Intersect/Intersect ALL/Except/Except ALL。涉及 ALL 的操作需要去重。常以 Append 为输入结点
	HashSetOp [type]	INTERCECT/EXCEPT（用 Hash 算法完成操作，不需要排序）。type 值同上。涉及 ALL 的操作需要去重。常以 Append 为输入结点

(续)

类 型	结 点	说 明
控制类型	Result	一次性结果获取（如子查询可以在查询优化阶段获得结果，多以 InitPlan 作为输入）
	ModifyTable	插入 / 修改 / 删除（Insert/Update/Delete 操作的 SQL）
	BitmapAnd	多个单独索引作为 AND 操作符的左右操作数
	BitmapOr	多个索引或一个索引的多个索引列作为 OR 操作符的左右操作数
	RecursiveUnion	用于处理递归定义的 UNION 语句（与 WorkTable Scan 配合使用）
过滤器	Filter	常规过滤器（对元组按条件进行过滤，完成选择操作）
	Join Filter	连接操作过滤器（Hash 连接、归并连接、嵌套循环连接操作情况下使用的过滤器）
	One-Time	一趟过滤器（对于能一次性获得结果的 Result 操作，一趟过滤即可满足过滤要求）
连接条件	Hash Cond	显示 Hash 连接的连接条件
	Merge Cond	显示 Merge 连接的连接条件
其他类型	Foreign Scan	外部表（对外部对象的访问）
	LockRows	查询中带有 FOR UPDATE/ FOR SHARE 的操作，需要对被操作对象加锁
	Remote SQL	配合 Foreign Scan 表示远程的 SQL
	InitPlan	与 Result 配合，在初始化查询计划的阶段即可完成的操作
	Recheck Cond	对条件重新进行检查
	Output	表示输出信息

C.2 MySQL 的查询执行计划解析

MySQL V5.6.10 版本使用 Explain 类表示查询执行计划。通过 EXPLAIN 这个 SQL 语句触发，显示查询执行计划。

C.2.1 EXPLAIN 功能

语法格式：

```
EXPLAIN [explain_type] explainable_stmt
```

可选项包括：

```
EXTENDED | PARTITIONS | FORMAT = format_name
format_name:
    TRADITIONAL | JSON
```

说明：

- ❑ EXPLAIN 命令：显示 SQL 语句的查询执行计划。
- ❑ EXPLAIN EXTENDED 命令：显示 SQL 语句的详细的查询执行计划；之后可以通过 SHOW WARNINGS 命令查看详细的信息。

2. 结点解析

MySQL 查询计划的输出列的含义如下。

- ❑ **id**：每个被独立执行的操作的标识，表示对象被操作的顺序；id 值大，先被执行；如果相同，执行顺序从上到下。
- ❑ **select_type**：查询中每个 select 子句的类型；具体值如表 C-2 所示。
- ❑ **table**：名字，被操作的对象名称，通常是表名，但有其他格式。
- ❑ **partitions**：匹配的分区信息（对于非分区表值为 NULL）。
- ❑ **type**：连接操作的类型；具体值如表 C-3 所示。
- ❑ **possible_keys**：备选的索引（列出可能被使用到的索引）。
- ❑ **key**：经优化器选定的索引；常用 ANALYZE TABLE 命令，可以使优化器正确地选择索引。
- ❑ **key_len**：被优化器选定的索引键的长度，单位是字节。
- ❑ **ref**：表示本行被操作的对象参照对象（被参照的对象可能是一个常量用 const 表示，也可能是其他表的 key 指向的对象）。
- ❑ **rows**：查询执行所扫描的元组个数（对于 InnoDB，此值是估计值）。
- ❑ **filtered**：按照条件表上数据被过滤的元组个数的百分比， $\text{rows} \times \text{filtered} / 100$ 可以求出过滤后的元组数即实际的元组数。
- ❑ **Extra**：MySQL 查询优化器执行查询的过程中对查询计划的重要补充信息，具体如表 C-4 所示。

表 C-2 MySQL 查询执行计划 select_type 列子句类型表

select_type	说 明
SIMPLE	简单的 SELECT 语句（不包括 UNION 操作或子查询操作）
PRIMARY	查询中最外层的 SELECT（如两表做 UNION 或存在有子查询，外层的表被作为 PRIMARY，内层被作为 UNION）
UNION	UNION 操作中，查询中处于内层的 SELECT（内层的 SELECT 语句与外层的 SELECT 语句没有依赖关系）
DEPENDENT UNION	UNION 操作中，查询中处于内层的 SELECT（内层的 SELECT 语句与外层的 SELECT 语句有依赖关系）
UNION RESULT	UNION 操作的结果，id 通常为 NULL
SUBQUERY	子查询中的首个 SELECT（如果有多个子查询存在）
DEPENDENT SUBQUERY	子查询中的首个 SELECT，但依赖于外层的表（如果有多个子查询存在）
DERIVED	被驱动的 SELECT 子查询（子查询位于 FROM 子句）
MATERIALIZED	被物化的子查询
UNCACHEABLE SUBQUERY	对于外层的主表，子查询不可被物化，每次都需要计算（耗时操作）
UNCACHEABLE UNION	UNION 操作中，内层的不可物化的子查询（类似于 UNCACHEABLE SUBQUERY）

表 C-3 连接操作数据访问方式表

代码表示方式 [⊖]	说 明
JT_SYSTEM	常量表情况一，表上只有一条元组匹配
JT_CONST	常量表情况二，WHERE 条件筛选后表上至多有一条元组匹配，如：WHERE table.pk = 2 (pk 列是主键列，值为 2 的要么有一条，要么没有)
JT_EQ_REF	参与连接运算的表，是内表（在代码实现的算法中，两表连接时作为循环中的内循环遍历对象，这样的表为内表）。基于索引（连接字段上存在唯一索引或主键索引，且操作符必须是“=”谓词，索引的值不能为 NULL）做扫描，使得对外表的一条元组，内表只有唯一一条元组与之对应
JT_REF	可用于单表扫描或连接。参与连接运算的表，是内表。基于索引（连接字段上的索引是非唯一索引，操作符必须是“=”谓词，连接字段值不可为 NULL）做扫描，使得对外表的一条元组，内表可有若干条元组与之对应
JT_REF_OR_NULL	类似 ref，只是搜索条件条件包括：连接字段的值可为 NULL 的情况。如：WHERE col = ... OR col IS NULL
JT_RANGE	范围扫描。基于索引做范围扫描，为诸如 BETWEEN、IN、>=、LIKE 类操作提供支持
JT_INDEX_SCAN	索引做扫描。是基于索引在索引的叶子结点上找满足条件的数据（不需要访问数据文件）
JT_ALL	全表扫描或范围扫描；不使用索引，顺序扫描，直接读取表上的数据（访问数据文件）
JT_UNIQUE_SUBQUERY	在子查询中，基于唯一索引进行扫描。类似于 eq_ref
JT_INDEX_SUBQUERY	在子查询中，基于除唯一索引之外的索引进行扫描
JT_INDEX_MERGE	多重范围扫描。两表连接的每个表的连接字段上均有索引存在且索引有序，结果合并在一起。适用于作集合的并、交操作
JT_FT	FT, FullText, 全文检索

表 C-4 Extra 信息解释表

Extra 值	说 明
Child of 'table' pushed join@1	MySQL Cluster 使用的参数。表示在连接操作中某个表被下推到 NDB 引擎的某个结点上执行
const row not found	在连接中，查询一个没有元组的空表
Deleting all rows	对于删除操作，某些存储引擎支持快速删除数据
Distinct	优化 DISTINCT 操作，在找到第一匹配的元组后即停止找同样值的动作
FirstMatch(tbl_name)	半连接的算法中对表采取了首次匹配的连接策略
Full scan on NULL key	子查询中，优化器不能使用基于索引的访问方式时（因有空值存在）所采取的一种数据访问策略
Impossible HAVING	HAVING 子句的值总是 FALSE（不能获取任何元组）
Impossible WHERE	WHERE 子句的值总是 FALSE（不能获取任何元组）
Materialize, Scan	MySQL 5.6.7 版本之前，表示使用单一被物化的临时表；之后的版本，物化是通过 select_type 列表达的

⊖ 代码中的表示方式可参考枚举类型 enum join_type。

(续)

Extra 值	说 明
No matching min/max row	形如 SELECT MIN(...) FROM ... WHERE condition 的查询中没有满足条件的元组（在查询执行计划生成阶段，MySQL 可以利用索引对此类查询进行判断）
no matching row in const table	连接操作中，表没有元组或者没有元组满足唯一索引对应的条件
No matching rows after partition pruning	对于 DELETE 或 UPDATE 操作，在分区剪枝后优化器找不到要操作的元组（条件不匹配）
No tables used	查询没有 FROM 子句或有 FROM DUAL 子句（如 EXPLAIN INSERT INTO t VALUES(10)）
Not exists	在左连接中，优化器可以优化满足“空值拒绝”的情况，这时，只要右表有一个元组满足连接条件，优化器则不检查右表中更多的行。例如，表 t1、t2，其中 t2.id 为 not null，对于 SELECT * FROM t1 LEFTJOIN t2 ON t1.id=t2.id WHERE t2.id IS NULL；由于 t2.id 非空，所以只可能是 t1 中有，而 t2 中没有，所以在 t2 中找到一次 t2.id 即可满足
Range checked for each record (index map: N)	优化器发现没有好的 index 可用，但发现如果进一步获取下一张要连接的表的列的值后，某些 index 可以通过 range 或 index_merge 获取行。这个过程不会快，但比没有任何索引时做表连接要快
Scanned N databases	在查询 INFORMATION_SCHEMA 模式中的对象时，读取数据时涉及的数据库个数，如 Scanned all databases 表示读取了所有库的信息，值为 0、1 或 all
Select tables optimized away	在没有 GROUPBY 子句的情况下，基于索引优化 MIN/MAX 操作，或者对于 MyISAM 存储引擎优化 COUNT(*) 操作，不必等到执行阶段再进行计算，查询执行计划生成的阶段即完成优化
Skip_open_table, Open_frm_only, Open_trigger_only, Open_full_table	查询 INFORMATION_SCHEMA 模式中的对象，相关的 4 项操作如下： Skip_open_table：不必打开表即可从数据库目录中获取到相应信息 Open_frm_only：只有表的 .frm 文件需要被打开 Open_trigger_only：只有表的 .TRG 文件需要被打开 Open_full_table：需要查找未被优化的信息，需要打开 .frm、.MYD、.MYI 文件以获取信息
Start materialize, End materialize, Scan	类似于 Materialize。在 MySQL 5.6.7 版本之前，表示使用了被物化的临时表；之后的版本，被物化的临时表用 select_type 表示
Start temporary, End temporary	半连接优化方式中，使用了重复淘汰（Duplicate Weedout）优化策略
unique row not found	在类似于 SELECT ... FROM tbl_name 查询中，没有满足唯一索引或主键索引的元组
Using filesort	使用文件完成排序操作
Using index	不需要读取数据文件从索引树（索引文件）中即可获得信息。 如果同时出现 Using where，表明索引被用来执行索引键值的查找；没有 Using where，表明索引被用来读取数据而非执行查找动作
Using index condition	为获取表的元组尝试通过访问索引来获取，即能用索引尽量用索引否则才进行全表扫描（索引被下推）
Using index for group-by	利用索引优化 GROUPBY 或 DISTINCT 操作，避免额外的磁盘操作

(续)

Extra 值	说 明
Using join buffer (Block Nested Loop) Using join buffer (Batched Key Access)	使用了连接缓存 Block Nested Loop: 连接算法是块嵌套循环连接 Batched Key Access: 连接算法是批量索引连接
Using MRR	MRR, Multi-Range Read Optimization, 使用了 MRR 技术优化随机读 (索引会造成随机读, MRR 可减缓索引的危害)
Using sort_union(...), Using union(...), Using intersect(...)	使用 index_merge 连接类型合并索引扫描(排序后合并、直接合并、 做差合并)
Using temporary	用临时表保存中间结果, 常用于 GROUP BY 和 ORDER BY 操作中
Using where	使用了 WHERE 子句(意在过滤元组)
Using where with pushed condition	条件下推到集群中的各个结点。对 NDB 存储引擎的表适用

附录 D

如何跟踪查询执行计划

PostgreSQL 和 MySQL 都提供了查询执行计划动态跟踪的机制，方便用户查看 SQL 语句的优化过程和方式，这为用户理解 PostgreSQL 和 MySQL 的查询优化器的实现方式提供了方便。

假定创建表并插入若干数据，命令如下：

```
CREATE TABLE t1 (id1 INT, a1 INT, b1 INT, PRIMARY KEY(id1));
CREATE TABLE t2 (id2 INT, a2 INT, b2 INT);
CREATE TABLE t3 (id3 INT UNIQUE, a3 INT, b3 INT);
CREATE TABLE t4 (id4 INT, a4 INT, b4 INT);
CREATE TABLE t5 (id5 INT UNIQUE, a5 INT, b5 INT);
```

D.1 跟踪 PostgreSQL 的查询优化过程

PostgreSQL 提供了跟踪查询优化过程的机制和一些可以设置的参数，下面具体分析。

D.1.1 查询优化跟踪的使用方式

PostgreSQL 提供了若干个参数，对查询优化器的执行过程进行跟踪，可以通过设置这些参数，掌握比查询执行计划更为详细的信息。

使用 SQL 语句，执行如下过程，即可以打开查询优化过程的跟踪机制：

```
postgres=# set debug_print_parse=on; // 打开语法分析跟踪
SET
postgres=# set debug_print_rewritten=on; // 打开查询重写跟踪
SET
postgres=# set debug_print_plan=on; // 打开执行计划生成跟踪
SET
```

在 PostgreSQL 服务器的输出中，显示如下内容：

LOG: parse tree: // 第一部分: 输出语法分析阶段的查询树的结果, 即 Query 的结果

```
DETAIL:  {QUERY
          :commandType 1
          :querySource 0
          :canSetTag true
          :utilityStmt <>
          :resultRelation 0
          :hasAggs false
          :hasWindowFuncs false
          :hasSubLinks false
          :hasDistinctOn false
          :hasRecursive false
          :hasModifyingCTE false
          :hasForUpdate false
          :cteList <>
          :rtable (
            {RTE
```

...

STATEMENT: select * from tkey;

LOG: rewritten parse tree: // 第二部分: 输出查询重写阶段的查询树的结果, 即重写后的 Query 的结果

```
DETAIL: (
        {QUERY
        :commandType 1
        :querySource 0
        :canSetTag true
        :utilityStmt <>
        :resultRelation 0
        :hasAggs false
        :hasWindowFuncs false
        :hasSubLinks false
        :hasDistinctOn false
        :hasRecursive false
        :hasModifyingCTE false
        :hasForUpdate false
        :cteList <>
        :rtable (
```

...

STATEMENT: select * from tkey;

LOG: plan: // 第三部分: 输出查询优化阶段的执行计划的结果, 即 PlannedStmt 的结果

```
DETAIL:  {PLANNEDSTMT
          :commandType 1
          :queryId 0
          :hasReturning false
          :hasModifyingCTE false
```

```
      :canSetTag true
      :transientPlan false
      :planTree
        {SEQSCAN
...

STATEMENT: select * from tkey;
```

D.1.2 与查询优化跟踪机制相关的参数

PostgreSQL 提供了会话级（session 级）的参数，用于跟踪查询优化各个阶段的过程产物，如表 D-1 所示。

表 D-1 MySQL 查询优化跟踪相关的参数表

参 数 名	参数含义与用法
debug_print_parse	输出语法分析阶段的查询树的结果，即 Query 的内容 布尔值，值为 ON 或 OFF，表示打开或关闭本项功能，默认值是 OFF 如：set debug_print_parse=on;
debug_print_rewritten	输出查询重写阶段的查询树的结果，即重写后的 Query 的内容 布尔值，值为 ON 或 OFF，表示打开或关闭本项功能，默认值是 OFF 如：set debug_print_rewritten=on;
debug_print_plan	输出查询优化阶段的执行计划的结果，即 PlannedStmt 结构体的内容 布尔值，值为 ON 或 OFF，表示打开或关闭本项功能，默认值是 OFF 如：set debug_print_plan=on

PostgreSQL 还提供了其他与查询优化技术相关的参数，用以调节查询优化器的执行过程或者算法选择，可参考相关资料深入掌握。

D.2 跟踪 MySQL 的查询优化过程

MySQL 提供了跟踪查询优化过程的机制和一些可以设置的参数，下面具体分析。

D.2.1 查询优化跟踪的使用方式

使用 SQL 语句，执行如下过程，即可以打开查询优化过程的跟踪机制：

```
set optimizer_trace_max_mem_size=300000; // 设置查询执行过程跟踪时使用的内存空间大小
set end_markers_in_json=true;           // 设置使用 JSON 格式输出的内容的结束标志
SET optimizer_trace="enabled=on";       // 设置参数 optimizer_trace
SELECT ...; # your query here             // 用户的查询语句
SELECT * FROM INFORMATION_SCHEMA.OPTIMIZER_TRACE; // 参阅优化过程的信息
SET optimizer_trace="enabled=off";       // 关闭参数 optimizer_trace
```

如果 INFORMATION_SCHEMA.OPTIMIZER_TRACE 中查询出的信息太多，则可以把信息输出到文件，命令如下：

SELECT TRACE INTO DUMPFILE "json.txt" FROM INFORMATION_SCHEMA.OPTIMIZER_TRACE;

示例 执行多表连接的查询，如图 D-1 所示。

```
mysql> EXPLAIN EXTENDED SELECT * FROM t3, <t2 LEFT JOIN t1 ON id1=id2> WHERE b3=b1 AND b2>10 AND a3 IN <SELECT b5 FROM t5 WHERE id1=id5> AND EXISTS <
SELECT 1 FROM t4 WHERE b4>5>;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	t3	ALL	NONE	NONE	NONE	NONE	100	100.00	NONE
1	PRIMARY	t2	ALL	NONE	NONE	NONE	NONE	100	100.00	Using where; Using join buffer <Block Nested Loop>
1	PRIMARY	t5	ref	id5	id5	5	test.t2.id2	1	100.00	Using where
1	PRIMARY	t1	eq_ref	PRIMARY	PRIMARY	4	test.t2.id2	1	100.00	Using where
3	SUBQUERY	t4	ALL	NONE	NONE	NONE	NONE	7	100.00	Using where

5 rows in set, 2 warnings (0.02 sec)

图 D-1 MySQL 多表连接查询执行计划

获取查询优化跟踪信息，语句如下：

SELECT trace FROM information_schema.optimizer_trace;

获取查询优化跟踪信息得到的结果摘录部分如下：

```
{
  "steps": [/* 表示步骤，主要阶段的步骤就 2 个，一个是 JOIN::prepare 方法，一个是 JOIN::optimizer 方法，如果有视图，则视图的信息在 join_preparation 之前 */
    {
      "join_preparation": { /* JOIN::prepare 方法阶段的优化 */
        "select#": 1,
        "steps": [
          {
            "join_preparation": {
              "select#": 2, /* 子查询 2 的优化 */
              "steps": [
                {
                  "expanded_query": "/* select#2 */ select `t5`.`b5` from `t5` where (`t1`.`id1` = `t5`.`id5`)"
                },
                {
                  "transformation": {
                    "select#": 2,
                    "from": "IN (SELECT)", /* 把子查询 2 的 IN 子查询转换为 EXISTS 操作 */
                    "to": "semijoin",
                    "chosen": true
                  }
                }
              ]
            }
          }
        ]
      }
    }
  ],
  {
    "join_preparation": {
```

```

        "select#": 3,
        "steps": [
            {
                "expanded_query": "/* select#3 */ select 1 from `t4` where (`t4`.`b4` > 5)"
            }
        ]
    },
    /* expanded_query, 表示关键阶段的优化结果 */
    "expanded_query": "/* select#1 */ select `t3`.`id3` AS `id3`,`t3`.`a3` AS `a3`,`t3`.`b3` AS `b3`,`t2`.`id2` AS
`id2`,`t2`.`a2` AS `a2`,`t2`
`.`b2` AS `b2`,`t1`.`id1` AS `id1`,`t1`.`a1` AS `a1`,`t1`.`b1` AS `b1` from `t3` join (`t2` left join `t1` on((`t1`.`id1` = `t2`.`id2`)))
where ((`t3`
`.`b3` = `t1`.`b1`) and (`t2`.`b2` > 10) and `t3`.`a3` in (/* select#2 */ select `t5`.`b5` from `t5` where (`t1`.`id1` = `t5`.`id5`)) and
exists(/* sel
ect#3 */ select 1 from `t4` where (`t4`.`b4` > 5)))"
    }
}
},
{
    "join_optimization": { /* JOIN::optimizer 阶段的优化 */
        "select#": 1,
        "steps": [
            {
                "transformation": {
                    "select#": 2,
                    "from": "IN (SELECT)",
                    "to": "semijoin",
                    "chosen": true,
                    "evaluating_constant_semijoin_conditions": [
                    ]
                }
            }
        ],
        {
            "transformations_to_nested_joins": { /* 发生在 simplify_joins 函数阶段 */
                "transformations": [
                    "semijoin", /* JOIN::optimizer 阶段执行了半连接优化 */
                    "outer_join_to_inner_join", /* JOIN::optimizer 阶段执行了外连接消除优化 */
                    "JOIN_condition_to_WHERE", /* JOIN::optimizer 阶段执行了连接条件转换到 WHERE 子句 */
                    "parenthesis_removal" /* JOIN::optimizer 阶段执行了嵌套消除优化, 去除嵌套的括号 */
                ],
                "expanded_query": "/* select#1 */ select `t3`.`id3` AS `id3`,`t3`.`a3` AS `a3`,`t3`.`b3` AS `b3`,`t2`.`id2` AS
`id2`,`t2`.`a2` AS `a2`,
`t2`.`b2` AS `b2`,`t1`.`id1` AS `id1`,`t1`.`a1` AS `a1`,`t1`.`b1` AS `b1` from `t3` semi join (`t5`) join `t2` join `t1` where ((`t3`.`b3`
= `t1`.`b1`)
and (`t2`.`b2` > 10) and 1 and exists(/* select#3 */ select 1 from `t4` where (`t4`.`b4` > 5)) and (`t1`.`id1` = `t5`.`id5`) and
(`t3`.`a3` = `t5`.`b
5`) and (`t1`.`id1` = `t2`.`id2`))"
            }
        ]
    }
}

```

```
    }
  },
  {
    "condition_processing": { /* 对条件进行处理的过程 */
      "condition": "WHERE",
... // 省略较多内容
      { /* 发生在 choose_table_order 函数阶段 */
        "considered_execution_plans": [
          {
            "plan_prefix": [
            ],
            "table": "`t4`",
            "best_access_path": {
              "considered_access_paths": [
                {
                  "access_type": "scan",
                  "rows": 7,
                  "cost": 2.4,
                  "chosen": true
                }
              ]
            },
            "cost_for_plan": 2.4,
            "rows_for_plan": 7,
            "chosen": true
          }
        ]
      },
... // 省略较多内容，还有一些与执行相关的信息
```

D.2.2 与查询优化跟踪机制相关的参数

MySQL 除了提供了会话级（session 级）的 optimizer_trace 参数外，还提供了其他与查询优化跟踪相关的会话级参数，如表 D-2 所示。

表 D-2 MySQL 查询优化跟踪相关的参数表

参 数 名	参数含义与用法
optimizer_trace	查询优化器跟踪开关，可以打开或关闭 格式为：option=val[,option=val...] 如：SET optimizer_trace="enabled=on";
optimizer_trace_features	查询优化器某个特性的跟踪开关，可以打开或关闭。默认是全部打开 格式为：name=val[,name=val...] name 的值可以为：greedy_search, range_optimizer, dynamic_range, repeated_subselect 之一 val 的值可以为：on、off、default 如：SET optimizer_trace_features="greedy_search=on";
optimizer_trace_limit	可以跟踪的最大 SQL 个数，默认值为 1

(续)

参 数 名	参数含义与用法
optimizer_trace_offset	可以跟踪的 SQL 的相对位置，默认值为 -1。与 optimizer_trace_offset 配合使用方法如下： offset=-1, limit=1：显示最近一次 trace offset=-1, limit=2：显示最近二次 trace offset=-2, limit=1：显示最近的前一个 trace offset=-3, limit=2：显示最近的前 2 个 trace
optimizer_trace_max_mem_size	设置 trace 机制保存跟踪信息时使用的内存最大容量，单位是字节。默认值为 16k

MySQL 还提供了其他与查询优化技术相关的参数，用以调节查询优化器的执行过程或者算法选择，可参考相关资料深入掌握。

电子书吧

QQ:

1759560190

定制各科电子书